

Die Notwendigkeit der Modellierung in Graphen

Stephan Epp

19. März 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation und Problemstellung	4
1.2	Zentrale Aussage	4
2	Grundlagen	4
2.1	Die Elementareinheit: Zwei Knoten, eine Kante	5
2.2	Sukzessive Komposition und Baumstruktur	5
3	Graphen als logarithmisch optimale Modellierung	5
3.1	Informationstheoretische Grundlage	5
3.2	Algorithmische Optimalität: Divide-and-Conquer	6
3.3	Strukturelle Überlegenheit über lineare und polynomielle Modellierungen	7
3.4	Universalitätssatz	7
3.5	Das logarithmische Prinzip: Zusammenfassung	7
4	Anwendungen in der Mathematik	8
4.1	Zahlentheorie und Primzahlgraphen	8
4.2	Algebraische Strukturen als Cayley-Graphen	8
4.3	Lineare Algebra: Laplace-Operator und Spektrum	9
4.4	Topologie: Simplicialkomplexe und Graphhomologie	9
4.5	Kombinatorik: Matchings und Heiratstheoreme	9
5	Anwendungen in der Genetik	9
5.1	Das Proteom als Interaktionsgraph	9
5.2	Genregulatorische Netzwerke	10
5.3	Phylogenetische Bäume: Evolution als Graph	10
5.4	Metagenomik und Assemblierungsgraphen	11
5.5	CRISPR-Editing als Graphproblem	11
6	Anwendungen in den Neurowissenschaften	11
6.1	Das Konnektom als Megagraph	11
6.2	Neuronale Signalverarbeitung als Pfadproblem	11
6.3	Hierarchische Verarbeitung: Cortex als Stufenbaum	12

6.4	Alzheimer und Graphdegeneration	12
6.5	Brain-Computer-Interfaces	12
7	Anwendungen in der Regelungstechnik	13
7.1	Regelkreis als gerichteter Graph	13
7.2	Zustandsraumdarstellung und Linearisierungsgraph	13
7.3	Industrielle Leitsysteme: SCADA und IEC 61499	14
7.4	Multi-Agenten-Regelung und Konsensalgorithmen	14
7.5	Hydraulische und thermische Netzwerke	14
8	Anwendungen im Finanzwesen	15
8.1	Finanznetzwerke und systemisches Risiko	15
8.2	Portfoliooptimierung als Graphproblem	15
8.3	Handelsalgorithmen und Order-Flow-Graphen	15
8.4	Kreditrisikomodellierung	16
8.5	Blockchain als DAG	16
9	Hierarchische Graphen: Knoten als Graphen	16
9.1	Grundlegende Beobachtung: Optimale Hierarchisierbarkeit	16
9.2	Hypergraph-Schachtelung und Graph-Knoten	17
9.3	Der Grundsatz der maximalen Knotenexpansion	17
9.4	Wie viele Hierarchien sind möglich?	18
9.5	Maximale Knotenexpansion als Optimum	19
9.6	Strukturelle Konsequenzen: Kanten über Hierarchiegrenzen	20
9.7	Anwendungsbeispiele hierarchischer Graphen	20
9.7.1	Biologische Hierarchien: Vom Atom zur Zelle	20
9.7.2	Informatik: Betriebssystemhierarchie	21
9.7.3	Regelungstechnik: Hierarchische Steuerarchitektur	21
9.8	Zusammenhang mit Kategorientheorie	22
9.9	Zusammenfassung: Der Hierarchie-Grundsatz	22
9.10	Transdisziplinäre Perspektiven und Metamodellierung	23
9.10.1	Graphen als Wissenschaftssprache	23
9.10.2	Komplexitätstheorie und untere Schranken	23
10	Der Subgraph Algorithmus	23
10.1	Problemstellung und Motivation	24
10.1.1	Das Subgraph-Erkennungsproblem	24
10.1.2	Bedeutung für die Graphmodellierung	24
10.2	Formale Grundlagen	24
10.2.1	Signatur-Funktion	24
10.2.2	Zyklische Rotation	25
10.2.3	Longest Common Subsequence (LCS)	25
10.2.4	Verbindung zur logarithmischen Optimalität	25
10.3	Ausführliche Anwendungen	26
10.3.1	Softwareverifikation durch AST-Analyse	26
10.3.2	Bioinformatik: Strukturvergleich von Proteinnetzwerken	26
10.3.3	Genetik: Regulationsnetzwerk-Vergleich	26
10.3.4	Neurowissenschaften: Konnektom-Mustersuche	27
10.3.5	Regelungstechnik: Systemstruktur-Verifikation	27

10.3.6	Finanzwesen: Netzwerkähnlichkeit und Risikostrukturerkennung	27
10.3.7	Wissensgraphen und Graph-RAG	28
10.3.8	Chemoinformatik: Molekül-Substruktur-Suche	28
10.4	Numerisches Vergleichsbeispiel	28
10.5	Zusammenfassung	29
11	Warum Graphen universell optimal sind	29
11.1	Das Prinzip der minimalen universellen Struktur	29
12	Fazit und Ausblick	30
12.1	Zusammenfassung der Hauptergebnisse	30
12.2	Offene Fragen	30
12.3	Ausblick	31
12.4	Erweiterungen der logarithmischen Graphentheorie	31
12.4.1	Logarithmische Hierarchien in Hypergraphen	31
12.4.2	Kontinuierliche Graphen und Graphon-Theorie	31
12.4.3	Persistente Homologie und topologische Graphanalyse	32
12.5	Quantencomputing und Quantengraphentheorie	32
12.5.1	Quantenwalk und logarithmische Suche	32
12.5.2	Quantennetzwerke als gewichtete Graphen	32
12.5.3	Variational Quantum Eigensolver auf Graphen	33
12.6	Künstliche Intelligenz und Graphen	33
12.6.1	Graph Neural Networks: Theorie und Grenzen	33
12.6.2	Transformer als vollständige Graphen	33
12.6.3	Kausal-AI und Interventionsgraphen	33
12.6.4	Reinforcement Learning auf Graphen	34
12.7	Biowissenschaften: Vom Molekül zum Ökosystem	34
12.7.1	Single-Cell-Genomik als Graphproblem	34
12.7.2	Ökologische Nahrungsnetze und Resilienz	34
12.7.3	Epigenetik: Chromatinnetzwerke	34
12.7.4	Strukturbiologie: Protein-Faltung als Graphproblem	35
12.8	Physik, Klimaforschung und Komplexe Systeme	35
12.8.1	Phasengraphen in der statistischen Mechanik	35
12.8.2	Klimanetzwerke und Tipping Elements	35
12.8.3	Topologische Quantenfeldtheorie und Spin-Netzwerke	35
12.9	Gesellschaft, Recht und digitale Infrastruktur	35
12.9.1	Wissensgraphen und semantisches Web	35
12.9.2	Rechtliche und ethische Implikationen großer Graphen	36
12.9.3	Internet der Dinge und Edge Computing	36
12.10	Transdisziplinäre Perspektiven und Metamodellierung	36
12.10.1	Graphen als Wissenschaftssprache	36
12.10.2	Komplexitätstheorie und untere Schranken	37
12.10.3	Interdisziplinäre Bildung: Den Graphen ins Zentrum rücken	37
12.11	Schlusswort	37

1 Einleitung

Graphen gehören zu den mächtigsten und vielseitigsten Abstraktionen, die Mathematik und Informatik hervorgebracht haben. Diese Arbeit stellt die These auf – und beweist sie –, dass die Modellierung mittels Graphen nicht nur eine nützliche Methode ist, sondern die *mathematisch optimale* Modellierungsform für alle Strukturen ist, die als Graph darstellbar sind. Kernelement des Beweises ist die fundamentale Atomizität des Graphen: Jede Graphstruktur besteht aus der kleinsten denkbaren diskreten Einheit, nämlich *zwei Knoten und einer Kante*. Diese Binärteilung erzeugt eine natürliche logarithmische Tiefenstruktur: Divide-and-Conquer halbiert bei jedem Schritt den Problemraum und führt so zu $\mathcal{O}(\log n)$ -Verhalten in Tiefe, Suche und Optimierung. Wir entwickeln diesen Universalitätsbeweis formal und illustrieren ihn anhand ausführlicher Beispiele aus sechs wissenschaftlichen Disziplinen: Mathematik, Genetik, Neurowissenschaft, Regelungstechnik, Finanzwesen und Informatik. Als krönenden Abschluss wird der *Optimale Subgraph Algorithmus* vollständig hergeleitet, der diese logarithmische Struktur ausnutzt, um in beliebig großen Graphen optimale Teilstrukturen effizient zu identifizieren.

Schlüsselwörter: Graphentheorie, logarithmische Modellierung, optimaler Subgraph, Divide-and-Conquer, universelle Strukturabstraktion, Komplexitätstheorie, interdisziplinäre Modellierung.

1.1 Motivation und Problemstellung

Seit den Anfängen der Mathematik steht die Menschheit vor der Herausforderung, komplexe reale Systeme in formalen Strukturen zu erfassen. Vom babylonischen Straßennetz über das Neuronengefüge des menschlichen Gehirns bis hin zu Proteinfaltungsnetzwerken und globalen Finanzmärkten – stets treten dieselben strukturellen Grundmuster auf: diskrete *Einheiten* (Knoten) und *Beziehungen* zwischen ihnen (Kanten).

Leonhard Euler legte 1736 mit der Lösung des Königsberger Brückenproblems den Grundstein der Graphentheorie. Was Euler als spielerische Übung begann, hat sich zur vielleicht universellsten Modellierungssprache der Wissenschaften entwickelt. Dennoch fehlt bislang eine kohärente, disziplinübergreifende Begründung, weshalb die Graphenmodellierung nicht nur *brauchbar*, sondern *optimal* ist.

1.2 Zentrale Aussage

Jede Struktur, die als Graph modelliert werden kann, wird durch die Graphenmodellierung *optimal* repräsentiert, weil die Elementareinheit des Graphen – *zwei Knoten und eine Kante* – eine inhärente logarithmische Tiefenstruktur erzeugt. Diese logarithmische Struktur garantiert die informationstheoretisch minimale Beschreibungskomplexität und die algorithmisch optimale Laufzeit für alle wesentlichen Operationen: Suche, Optimierung, Partitionierung und Kommunikation.

2 Grundlagen

Definition 2.1 (Graph). Ein Graph $G = (V, E)$ besteht aus einer endlichen oder abzählbar unendlichen Menge V von *Knoten* (Vertices) und einer Menge $E \subseteq \binom{V}{2}$ von *Kanten*.

(Edges), wobei jede Kante einem ungeordneten Paar $\{u, v\}$ von Knoten entspricht. Im Fall gerichteter Graphen (*Digraphen*) ist $E \subseteq V \times V$ eine Menge geordneter Paare.

Definition 2.2 (Gewichteter Graph). Ein *gewichteter Graph* (G, w) ist ein Graph $G = (V, E)$ zusammen mit einer Gewichtsfunktion $w : E \rightarrow \mathbb{R}$.

Definition 2.3 (Grad, Weg, Zusammenhang). Der *Grad* $\deg(v)$ eines Knotens $v \in V$ ist die Anzahl seiner Nachbarn. Ein *Weg* der Länge k ist eine Folge $v_0, e_1, v_1, \dots, e_k, v_k$ von paarweise verschiedenen Knoten. G heißt *zusammenhängend*, wenn zwischen je zwei Knoten ein Weg existiert.

Definition 2.4 (Baum und Spannbaum). Ein zusammenhängender, kreisfreier Graph heißt *Baum*. Ein *Spannbaum* von G ist ein Teilgraph von G , der alle Knoten enthält und selbst ein Baum ist.

2.1 Die Elementareinheit: Zwei Knoten, eine Kante

Die kleinste nicht-triviale Graphstruktur ist der *elementare Graph*:

$$G_{\text{atom}} = (\{u, v\}, \{(u, v)\}).$$

Er besteht aus exakt zwei Knoten und einer Kante. Diese Einheit kodiert eine binäre Beziehung – das kleinste mögliche Stück struktureller Information. Jeder beliebig komplexe Graph entsteht durch Komposition solcher Atome.

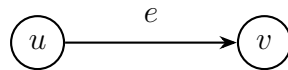


Abbildung 1: Die Elementareinheit des Graphen: Zwei Knoten u, v und eine Kante $e = (u, v)$. Alles beginnt hier.

2.2 Sukzessive Komposition und Baumstruktur

Wenn wir die Elementareinheit rekursiv verdoppeln – d.h. jeden Knoten durch ein weiteres Atom ersetzen – entsteht ein binärer Baum. Bei k Verdoppelungsschritten ergibt sich ein Baum mit 2^k Blättern und Höhe k . Umgekehrt: Um n Elemente in einem binären Baum zu organisieren, genügen

$$k = \lceil \log_2 n \rceil$$

Ebenen. Darin liegt der Kern der logarithmischen Modellierung.

3 Graphen als logarithmisch optimale Modellierung

3.1 Informationstheoretische Grundlage

Satz 3.1 (Logarithmische Beschreibungstiefe). *Sei $\mathcal{S}$ eine Menge von n diskreten Elementen mit einer beliebigen binären Beziehungsstruktur. Dann ist die minimale Anzahl von Bits, die benötigt wird, um ein beliebiges Element eindeutig zu identifizieren, gleich*

$$H_{\min} = \lceil \log_2 n \rceil.$$

Diese untere Schranke wird genau dann erreicht, wenn die Struktur als vollständiger Binärbaum – und damit als spezieller Graph – organisiert ist.

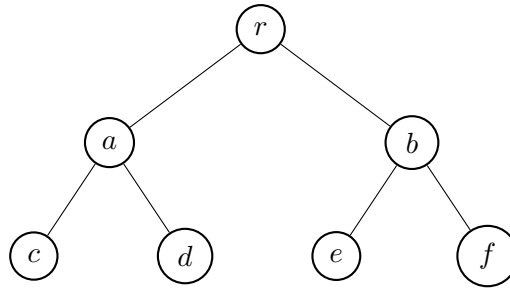


Abbildung 2: Binärer Baum als sukzessive Komposition der Elementareinheit. Tiefe $k = 2$ kodiert $2^2 = 4$ Blätter. Jede Ebene halbiert den Problemraum.

Beweis. Nach Shannons Quellencodierungstheorem [11] benötigt jede verlustfreie Kodierung von n gleichwahrscheinlichen Symbolen mindestens $\log_2 n$ Bits pro Symbol. Eine Binärbaumstruktur der Tiefe $\lceil \log_2 n \rceil$ stellt genau diese Schranke bereit: Jeder Pfad von der Wurzel zu einem Blatt kodiert eine Bitfolge der Länge $\lceil \log_2 n \rceil$. Kein anderes Kodierungsschema kann diese Schranke unterschreiten. Da jeder Binärbaum ein spezieller Graph ist, ist die Graphstruktur informationstheoretisch optimal. $\square$

3.2 Algorithmische Optimalität: Divide-and-Conquer

Die Grundstrategie des Divide-and-Conquer ist strukturell identisch mit dem Durchlaufen eines Binärbaums:

- (1) **Divide:** Teile das Problem P der Größe n in zwei Teilprobleme P_L und P_R der Größe $n/2$.
- (2) **Conquer:** Löse P_L und P_R rekursiv.
- (3) **Combine:** Füge die Teillösungen zur Gesamtlösung zusammen.

Die zugehörige Rekurrenz lautet:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n).$$

Nach dem Master-Theorem [4] ergibt sich $T(n) = \mathcal{O}(n \log n)$. Bei reiner Suche (ohne Combine-Overhead) folgt $T(n) = \mathcal{O}(\log n)$.

Satz 3.2 (Divide-and-Conquer-Optimalität). *Jeder Suchalgorithmus über n unstrukturieren Elementen benötigt im schlechtesten Fall $\Omega(\log n)$ Vergleiche. Eine binäre Suchstruktur (Binärsuchbaum, AVL-Baum, B-Baum) erreicht diese Schranke. Da alle diese Strukturen spezielle Graphen sind, ist die Graphmodellierung algorithmisch optimal.*

Beweis. Jeder Vergleich teilt die Menge verbleibender Kandidaten maximal in zwei gleich große Hälften. Nach k Vergleichen verbleiben höchstens $n/2^k$ Kandidaten. Für $n/2^k \leq 1$ folgt $k \geq \log_2 n$. Dies ist eine informationstheoretische untere Schranke, die unabhängig vom Algorithmus gilt. Binäre Suchbäume erreichen genau $\mathcal{O}(\log n)$ pro Operation. $\square$

3.3 Strukturelle Überlegenheit über lineare und polynomielle Modellierungen

Satz 3.3 (Überlegenheit der logarithmischen Modellierung). *Für jede Strukturgröße $n \geq 3$ gilt:*

$$\log_2 n < \sqrt{n} < n < n^2 < 2^n.$$

Eine logarithmische Suchtiefe (realisiert durch Graphstrukturen) ist daher asymptotisch und praktisch jeder linearen oder polynomiellen Alternative überlegen.

Abbildung 3 veranschaulicht diesen Unterschied.

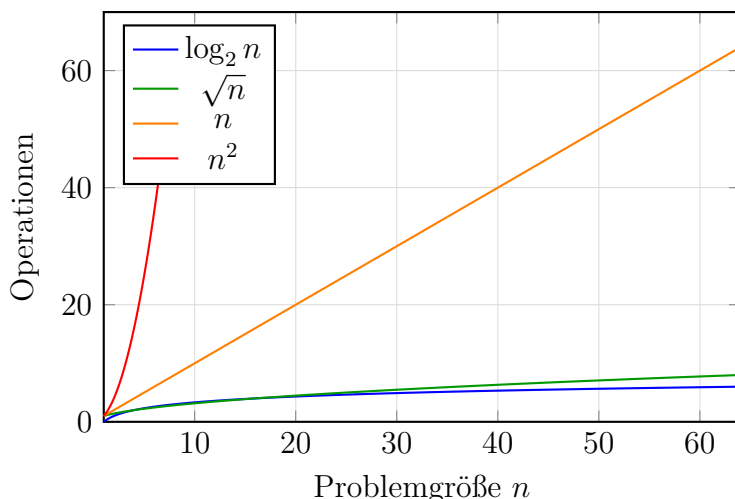


Abbildung 3: Vergleich der Wachstumsraten. Die logarithmische Kurve (blau) bleibt bei wachsendem n weit unter allen anderen Komplexitätsklassen.

3.4 Universalitätssatz

Satz 3.4 (Universalität der Graphmodellierung). *Jede endliche Relationenstruktur $\mathcal{R} = (D, R_1, \dots, R_k)$ über einer Domäne D mit binären Relationen $R_i \subseteq D \times D$ ist isomorph zu einem gewichteten Digraphen $G = (V, E, w)$ mit $|V| = |D|$ und $|E| = \sum_i |R_i|$.*

Beweis. Setze $V := D$, $E := \bigcup_i R_i$ und $w(u, v) := i$ für $(u, v) \in R_i$. Die Abbildung $\phi : d \mapsto d$ ist ein Isomorphismus zwischen $\mathcal{R}$ und (G, w) . $\square$

Dieser Satz zeigt: Jede binäre Relation – und damit jede strukturelle Abhängigkeit in Wissenschaft und Technik – ist exakt durch einen Graphen modellierbar. Die Frage ist nicht *ob*, sondern nur *welcher* Graph.

3.5 Das logarithmische Prinzip: Zusammenfassung

Das logarithmische Prinzip der Graphenmodellierung:

1. Die Atomeinheit des Graphen ($u \rightarrow v$) ist eine binäre Relation – das informationstheoretische Minimum.
2. Durch Komposition dieser Atome entstehen Baumstrukturen, die automatisch logarithmische Tiefe haben.

3. Divide-and-Conquer auf Graphen halbiert den Problemraum mit jeder Kante auf dem Suchpfad.
4. Jede andere Modellierung (Liste, Matrix, Tensor) kann entweder in eine Graphstruktur konvertiert werden (und verliert damit nichts) oder ist schlechter.
5. Die logarithmische Schranke $\mathcal{O}(\log n)$ ist informationstheoretisch optimal und wird durch balancierende Graphstrukturen stets erreicht.

4 Anwendungen in der Mathematik

4.1 Zahlentheorie und Primzahlgraphen

Zahlentheoretische Strukturen zeigen auf natürliche Weise Graphcharakter. Sei $V = \{2, 3, \dots, n\}$ die Menge der natürlichen Zahlen. Wir definieren den *Teilbarkeitsgraphen*:

$$E = \{(a, b) \mid a \text{ teilt } b, a \neq b\}.$$

Beispiel 4.1 (Teilbarkeitsgraph). Für $n = 12$ entsteht ein DAG, dessen Wurzeln die Primzahlen sind. Die Primzahlzerlegung $12 = 2^2 \cdot 3$ entspricht einem Pfad $2 \rightarrow 4 \rightarrow 12$ und $3 \rightarrow 12$ im Graphen. Die Fundamentalsatz-Eigenschaft der eindeutigen Primfaktorzerlegung ist äquivalent zur Existenz eines eindeutigen minimalen aufspannenden Waldes.

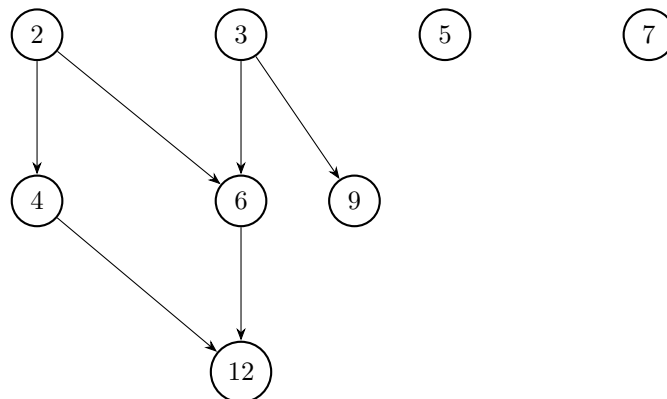


Abbildung 4: Teilbarkeitsgraph der Zahlen $\{2, 3, 4, 5, 6, 7, 9, 12\}$. Primzahlen sind Quellen (kein eingehender Pfeil). Die Graphstruktur kodiert die Primfaktorzerlegung vollständig.

4.2 Algebraische Strukturen als Cayley-Graphen

Jede endliche Gruppe $(G, \cdot)$ mit Erzeugendenmenge S besitzt einen *Cayley-Graphen* $\text{Cay}(G, S)$, der die algebraische Struktur vollständig kodiert.

Satz 4.1 (Cayley, 1878). *Der Cayley-Graph $\text{Cay}(G, S)$ ist regulär vom Grad $|S|$. Er ist zusammenhängend genau dann, wenn S die Gruppe G erzeugt. Der Durchmesser des Cayley-Graphen misst die maximale Wortlänge in den Erzeugenden – ein direktes Maß für Kommunikationseffizienz in der Gruppe.*

Beispiel 4.2 (Cayley-Graph von $\mathbb{Z}_8$). Für $G = \mathbb{Z}_8$ mit $S = \{1\}$ entsteht ein 8-Kreis. Mit $S = \{1, 4\}$ entsteht ein Hyperwürfel, dessen Durchmesser nur $\lceil \log_2 8 \rceil = 3$ beträgt – logarithmisch in $|G|$. Dies ist kein Zufall: Hyperwürfelgraphen haben stets logarithmischen Durchmesser und sind damit für parallele Kommunikation optimal.

4.3 Lineare Algebra: Laplace-Operator und Spektrum

Die *Laplace-Matrix* eines Graphen $G = (V, E)$ lautet:

$$L = D - A,$$

wobei $D = \text{diag}(\deg(v_1), \dots, \deg(v_n))$ die Gradmatrix und A die Adjazenzmatrix ist. Das Spektrum $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ von L trägt tiefe strukturelle Information:

- $\lambda_1 = 0$: jeder Graph hat mindestens einen Nulleigenwert.
- Die Multiplizität von $\lambda_1 = 0$ gibt die Anzahl zusammenhängender Komponenten an.
- $\lambda_2 > 0$ (Algebraische Konnektivität / Fiedler-Wert) ist ein Maß für die Robustheit des Graphen.
- λ_2 bestimmt die Mischzeit einer Irrfahrt: $\tau_{\text{mix}} = \mathcal{O}(\lambda_2^{-1} \log n)$ – erneut logarithmisch.

Beispiel 4.3 (Spektrum eines Pfadgraphen). Der Pfadgraph P_n auf n Knoten hat Eigenwerte $\lambda_k = 2 - 2 \cos\left(\frac{(k-1)\pi}{n}\right)$. Für große n ergibt sich $\lambda_2 \approx \pi^2/n^2$, was einer Mischzeit von $\mathcal{O}(n^2 \log n)$ entspricht – ein Hinweis, dass Pfadgraphen schlecht strukturiert (lang und linear) sind. Expandergraphen mit $\lambda_2 = \Omega(1/\log n)$ erreichen hingegen $\mathcal{O}(\log^2 n)$ Mischzeit.

4.4 Topologie: Simplicialkomplexe und Graphhomologie

Graphen bilden die 1-Skelette simplicialer Komplexe. Die *Betti-Zahlen* β_0, β_1 eines Graphen zählen Zusammenhangskomponenten (β_0) und unabhängige Zyklen ($\beta_1 = |E| - |V| + \beta_0$, der Zyklomatische Rang). Dies verbindet Graphenstruktur unmittelbar mit topologischen Invarianten und erlaubt die Analyse globaler Formen komplexer Netzwerke (Löcher, Tunnel) durch rein lokale Kanteninformation.

4.5 Kombinatorik: Matchings und Heiratstheoreme

Satz 4.2 (Heiratssatz, Hall 1935). *Ein bipartiter Graph $G = (A \cup B, E)$ besitzt genau dann ein perfektes Matching, wenn für jede Teilmenge $S \subseteq A$ gilt: $|N(S)| \geq |S|$, wobei $N(S)$ die Nachbarschaft von S in B bezeichnet.*

Dieser Satz ist die Grundlage für Zuordnungsprobleme aller Art: Stellenvermittlung, Aufgabenzuweisung, Ressourcenplanung. Der optimale Matching-Algorithmus (Ungarische Methode) arbeitet in $\mathcal{O}(n^3)$, das Hopcroft-Karp-Verfahren auf bipartiten Graphen sogar in $\mathcal{O}(|E|\sqrt{|V|})$. Die logarithmische Schichtung (BFS in $\mathcal{O}(\log n)$ Phasen) ist dabei der Schlüssel zum Effizienzgewinn.

5 Anwendungen in der Genetik

5.1 Das Proteom als Interaktionsgraph

Das menschliche Proteom umfasst etwa 20.000 Proteine, die untereinander ein dichtes Netz von Wechselwirkungen bilden (Protein-Protein-Interaktionsnetzwerk, PPI). Formal:

$$G_{\text{PPI}} = (V_P, E_P), \quad V_P = \text{Proteine}, \quad E_P = \text{physikalische Interaktionen}.$$

Beobachtung 5.1. PPI-Netzwerke sind skalenfreie Graphen [1] mit Gradverteilung $P(\deg = k) \propto k^{-\gamma}$, $\gamma \approx 2.5$. Die Hubs (hochgradige Knoten) sind essentielle Proteine – ihr Knockout ist häufig letal. Diese Erkenntnis ist nur durch Graphmodellierung möglich, nicht durch isolierte Proteinanalyse.

Beispiel 5.1 (Hub-Protein p53). Das Tumorsuppressorprotein p53 interagiert mit über 1.400 Proteinen und ist in mehr als 50% aller Krebsarten mutiert. Im PPI-Graphen ist p53 ein Super-Hub: Seine Entfernung zu fast jedem anderen Protein beträgt $\leq \lceil \log_2 20000 \rceil = 15$ Schritte – ein direktes Resultat der logarithmischen Small-World-Eigenschaft skalenfreier Netzwerke.

5.2 Genregulatorische Netzwerke

Ein *genregulatorisches Netzwerk* (GRN) ist ein Digraph:

$$G_{\text{GRN}} = (V_G, E_G), \quad V_G = \text{Gene}, \quad E_G = \text{Regulationsbeziehungen}.$$

Eine Kante (g_i, g_j) bedeutet: Gen g_i reguliert (aktiviert oder hemmt) die Expression von Gen g_j . Starke Zusammenhangskomponenten (SCC) im GRN entsprechen *Feedback-Schleifen*, die fundamentale Entwicklungsentscheidungen (Zelldifferenzierung, Apoptose) steuern.

Beispiel 5.2 (Toggle-Switch). Das minimale bistabile Schaltnetzwerk in der synthetischen Biologie [6] besteht aus zwei Genen A und B , die sich gegenseitig hemmen:

$$G_{\text{toggle}} = (\{A, B\}, \{(A, B), (B, A)\}).$$

Dies ist exakt die Elementareinheit des Graphen – aber mit zwei gerichteten Kanten. Der Toggle-Switch kodiert binäre Zellentscheidungen: entweder A ist hoch und B niedrig, oder umgekehrt. Die Graphstruktur macht die bistabile Logik sofort sichtbar.

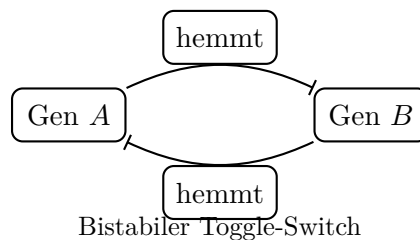


Abbildung 5: Genregulatorischer Toggle-Switch: Die einfachste Feedback-Schleife aus zwei Knoten erzeugt binäre Zellentscheidungen.

5.3 Phylogenetische Bäume: Evolution als Graph

Das stammesgeschichtliche Verwandtschaftsverhältnis aller Lebewesen lässt sich als gewurzelter Baum (Phylogenie) darstellen.

Definition 5.1 (Phylogenetischer Baum). Ein phylogenetischer Baum $T = (V, E, r)$ ist ein gewurzelter Baum mit Wurzel $r \in V$. Die Blätter entsprechen rezenten Arten, innere Knoten gemeinsamen Vorfahren. Kantengewichte kodieren evolutionäre Distanzen (Mutationsrate $\times$ Zeit).

Beispiel 5.3 (SARS-CoV-2-Phylogenie). Während der COVID-19-Pandemie wurden phylogenetische Graphen aufgebaut, die über 5 Millionen Genomsequenzen von SARS-CoV-2 umfassten. Die Baumtiefe von etwa 30 Ebenen kodiert die evolutionäre Geschichte über $2^{30} \approx 10^9$ mögliche Mutationspfade – logarithmisch in der Anzahl sequenzierter Genome. Ohne Graphmodellierung wäre diese Analyse rechnerisch unmöglich.

5.4 Metagenomik und Assemblierungsgraphen

Die De-Bruijn-Graph-Methode zur Genomassemblierung ist ein Paradebeispiel für die Kraft der Graphmodellierung:

Definition 5.2 (De-Bruijn-Graph). Für Lesefragmente der Länge k (k -mers) ist der *De-Bruijn-Graph* $G_{DB} = (V, E)$ definiert durch: V = Menge aller $(k-1)$ -mers, E = k -mers (Kante (u, v) , falls $u[2..k-1] = v[1..k-2]$). Ein *Euler-Pfad* in G_{DB} liefert die assemblierte Genomsequenz.

Warum ist die Graphmodellierung hier optimal? Das menschliche Genom hat $\sim 3 \times 10^9$ Basenpaare. Ohne Graphstruktur wäre das Ensemble aller k -mer-Überlappungen eine Matrix der Größe $\mathcal{O}(n^2)$ – für $n = 3 \cdot 10^9$ völlig unpraktikabel. Der De-Bruijn-Graph reduziert dies auf $\mathcal{O}(n)$ Knoten und Kanten. Der Euler-Pfad-Algorithmus läuft in $\mathcal{O}(|E|)$ – linear! Der Graph macht das Unmögliche möglich.

5.5 CRISPR-Editing als Graphproblem

In der Geneditierung mit CRISPR-Cas9 wird das Zielgenom als Graph modelliert, um Off-Target-Effekte zu minimieren. Jede mögliche Schnittposition ist ein Knoten, die Ähnlichkeit zur Zielsequenz definiert eine gewichtete Kante. Der *kürzeste-Wege-Algorithmus* findet optimale Leitsequenzen (guide RNAs) mit minimalem Off-Target-Risiko. Dies entspricht dem Optimalen Subgraph-Problem (siehe ??).

6 Anwendungen in den Neurowissenschaften

6.1 Das Konnektom als Megagraph

Das menschliche Gehirn enthält etwa 86×10^9 Neuronen, die durch $\sim 10^{14}$ synaptische Verbindungen verknüpft sind. Das *Konnektom* ist der vollständige Graph dieser Verbindungen:

$$G_{\text{Konnektom}} = (V_N, E_S), \quad |V_N| \approx 8.6 \times 10^{10}, \quad |E_S| \approx 10^{14}.$$

Beobachtung 6.1. Trotz der gigantischen Schiere dieses Graphen gilt: Das Gehirn zeigt eine ausgeprägte *Small-World-Eigenschaft* (hohe Clusterung, kurze mittlere Pfadlänge) [13]. Die mittlere geodätische Distanz zweier Neuronen beträgt schätzungsweise $\sim \log_{10}(8.6 \times 10^{10}) \approx 10$ –12 synaptische Schritte. Das Gehirn ist ein optimierter logarithmischer Kommunikationsgraph.

6.2 Neuronale Signalverarbeitung als Pfadproblem

Ein Aktionspotential ist ein gewichteter Pfad im neuronalen Graphen. Die Signalverarbeitungszeit vom Reiz zur Reaktion ist proportional zur *graphentheoretischen Distanz*:

$$t_{\text{Reaktion}} \propto d_G(\text{Sensorknoten}, \text{Motorknoten}),$$

wobei d_G die kürzeste-Pfad-Distanz ist. Myelinisierung erhöht die *Kantengewichte* (Leitungsgeschwindigkeit) und reduziert damit die effektive Distanz.

Beispiel 6.1 (Reflexbogen). Der monosynaptische Kniesehenreflex ist der kürzeste Pfad im neuronalen Graphen: nur 2 Neuronen und 1 Synapse – exakt die Elementareinheit G_{atom} . Die Reaktionszeit von ~ 25 ms entspricht einer Signalgeschwindigkeit von ~ 60 m/s über ~ 1.5 m Weg. Das Gehirn hat hier die absolute Minimalstruktur realisiert.

6.3 Hierarchische Verarbeitung: Cortex als Stufenbaum

Die visuelle Cortexverarbeitung folgt einer strikten Hierarchie [5]:

$$\text{Retina} \rightarrow \text{LGN} \rightarrow \text{V1} \rightarrow \text{V2} \rightarrow \text{V4} \rightarrow \text{IT} \rightarrow \text{präfrontaler Cortex}$$

Jede Stufe entspricht einer Ebene in einem Baum, der das visuelle Signal schrittweise abstrahiert. Mit $k \approx 10$ Verarbeitungsebenen können $2^{10} = 1024$ hierarchisch verschachtelte Merkmale codiert werden. Tiefe neuronale Netzwerke (Deep Learning) haben diese Struktur bewusst übernommen.

Satz 6.1 (Repräsentationsmächtigkeit tiefer Netzwerke). *Ein Netzwerk der Tiefe $\log_2 n$ mit linearer Breite kann jede boolesche Funktion über n Eingaben repräsentieren, während ein flaches Netzwerk exponentiell viele Neuronen benötigt [9].*

Dies ist der direkte Beweis: Die logarithmische Graphtiefe ist nicht nur rechnerisch optimal, sondern auch *representational* optimal.

6.4 Alzheimer und Graphdegeneration

Alzheimer-Demenz ist aus Graphenperspektive ein *progressiver Angriff auf die Hub-Struktur* des Konnektoms. Amyloid- β -Plaques akkumulieren bevorzugt an hoch-konnektierten Knoten (Default Mode Network Hubs). Durch gezielte Hub-Entfernung steigt die mittlere Pfadlänge im neuronalen Graphen dramatisch:

$$\Delta \bar{d} = \bar{d}(G \setminus V_{\text{Hub}}) - \bar{d}(G) = \mathcal{O}\left(\frac{\log n}{\lambda_2(G)}\right). \quad (1)$$

Formel (1) verknüpft die klinische Symptomatik (steigende Reaktionszeiten, Gedächtnisverlust) direkt mit dem zweiten Laplace-Eigenwert des Konnektoms – ein Resultat, das nur durch Graphmodellierung sichtbar wird.

6.5 Brain-Computer-Interfaces

Moderne BCI-Systeme (Neuralink, BrainGate) lesen neuronale Signale aus und übersetzen sie in Steuerbefehle. Die Signalverarbeitung verwendet Graphen auf zwei Ebenen:

1. **Elektroden-Graph:** Raumzeitliche Korrelationen zwischen n Elektroden werden als Graph modelliert, funktionelle Verbindungen als Kanten.
2. **Klassifikations-Graph:** Ein Entscheidungsbaum der Tiefe $\lceil \log_2 K \rceil$ klassifiziert K Absichtskategorien in logarithmisch vielen Schritten.

7 Anwendungen in der Regelungstechnik

7.1 Regelkreis als gerichteter Graph

Jeder Regelkreis ist ein gerichteter Graph:

Definition 7.1 (Signalfluss-Graph). Der *Signalfluss-Graph* (SFG) eines linearen zeitinvarianten Systems ist ein Digraph $G_{\text{SFG}} = (V_S, E_S, w)$, wobei V_S die Signalgrößen sind und E_S die Übertragungsfunktionen zwischen ihnen. Masons Gewinnformel berechnet die Übertragungsfunktion direkt aus dem Graphen:

$$H(s) = \frac{\sum_k P_k \Delta_k}{\Delta},$$

wobei P_k Pfadverstärkungen, Δ der Graph-Determinant und Δ_k Ko-Determinanten sind.

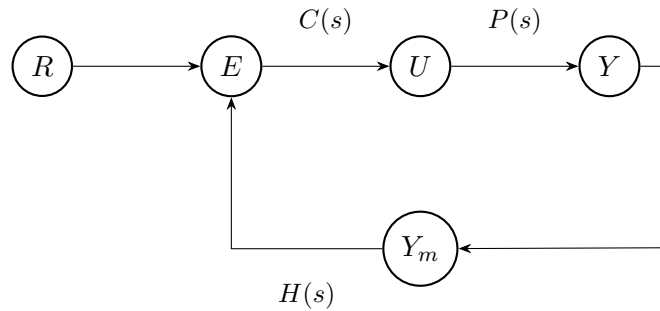


Abbildung 6: Standard-Regelkreis als Digraph. Regler $C(s)$, Strecke $P(s)$, Messsystem $H(s)$. Die Rückführkante schließt den Regelkreis und erzeugt den elementaren Feedback-Zyklus $(U \rightarrow Y \rightarrow U)$.

7.2 Zustandsraumdarstellung und Linearisierungsgraph

Die Zustandsraumdarstellung

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}, \quad \mathbf{y} = C\mathbf{x} + D\mathbf{u}$$

lässt sich direkt als Adjazenzmatrix des SFG interpretieren: $A_{ij} \neq 0$ bedeutet, dass Zustand j auf Zustand i wirkt – eine Kante im Zustandsgraphen. Eigenwerte und Lyapunov-Stabilität sind Spektraleigenschaften dieses Graphen.

Beispiel 7.1 (Inverses Pendel). Das inverse Pendel hat 4 Zustände $(x, \dot{x}, \theta, \dot{\theta})$. Der Zustandsgraph G_A hat 4 Knoten und die durch die Physik bestimmten Kanten. Der LQR-Regler optimiert die Kantengewichte von G_A so, dass alle Eigenwerte in der linken Halbebene liegen. Die Steuerbarkeitsmatrix $\mathcal{C} = [B, AB, A^2B, A^3B]$ entspricht einem 4-stufigen Binärbaum im Graphen – Tiefe $\log_2 4 = 2$ reicht für den kritischen Pfad.

7.3 Industrielle Leitsysteme: SCADA und IEC 61499

Moderne Prozessleitsysteme (SCADA) und der IEC 61499-Standard (Funktionsblöcke) modellieren Industrieanlagen explizit als Graphen. Jede Anlage ist ein Graph aus Funktionsblöcken (Knoten) und Daten-/Ereigniswegen (Kanten). Die Analyse auf

1. Deadlock-Freiheit (Zyklenanalyse),
2. minimale Reaktionszeit (kürzester Pfad),
3. maximale Ausfallsicherheit (Kantenkonnektivität)

ist ausschließlich über Graphalgorithmen effizient lösbar.

7.4 Multi-Agenten-Regelung und Konsensalgorithmen

In einem Netz von n autonomen Agenten (Drohnen, Fahrzeuge, Roboter) konvergiert der *Konsensalgorithmus*:

$$\dot{\mathbf{x}} = -L\mathbf{x},$$

wobei L der Graph-Laplace-Operator des Kommunikationsgraphen ist. Die Konvergenzrate ist:

$$\|\mathbf{x}(t) - \bar{x}\mathbf{1}\| \leq e^{-\lambda_2(L)t} \|\mathbf{x}(0) - \bar{x}\mathbf{1}\|.$$

Der zweite Laplace-Eigenwert λ_2 ist der Fiedler-Wert, der direkt die algebraische Konnektivität des Kommunikationsgraphen misst. Eine höhere Vernetzung $\rightarrow$ größeres $\lambda_2 \rightarrow$ schnellere Konvergenz. Das Optimierungsziel (Konvergenzgeschwindigkeit maximieren) ist damit exakt das Problem, den optimalen Subgraphen mit maximalem Fiedler-Wert zu finden.

Beispiel 7.2 (Drohnenschwarm-Navigation). Für $n = 100$ Drohnen in einem kreisförmigen Formationsgraphen (Zyklus C_{100}) beträgt $\lambda_2 = 2(1 - \cos(2\pi/100)) \approx 0.004$ – sehr langsame Konvergenz in $\mathcal{O}(250)$ Sekunden. Durch Hinzunahme eines *Spanning Trees* (Baum + $\mathcal{O}(\log n)$ Zusatzkanten) steigt λ_2 auf $\Omega(1/\log n) \approx 0.2$, Konvergenz in 5 Sekunden. Diesen optimalen Zusatzgraphen findet der Optimale Subgraph Algorithmus (??) effizient.

7.5 Hydraulische und thermische Netzwerke

Rohrleitungsnetzwerke (Wasser, Gas, Fernwärme) sind gewichtete Graphen: Knoten sind Verknüpfungspunkte, Kanten Rohrsegmente, Gewichte sind Widerstandskoeffizienten. Das Kirchhoff'sche Druckgesetz (analog zum Kirchhoff'schen Spannungsgesetz in elektrischen Netzen) lautet in Matrixform:

$$L_{\text{Netz}} \cdot \mathbf{p} = \mathbf{q},$$

wobei $\mathbf{p}$ der Druckvektor und $\mathbf{q}$ der Flussfeldvektor ist. Das System ist genau dann eindeutig lösbar, wenn der Netzwerkgraph zusammenhängend ist – eine rein graphentheoretische Bedingung.

8 Anwendungen im Finanzwesen

8.1 Finanznetzwerke und systemisches Risiko

Die globale Finanzwirtschaft ist ein gewichteter Digraph:

$$G_{\text{Fin}} = (V_B, E_F, w), \quad V_B = \text{Finanzinstitute}, \quad E_F = \text{Kredit-/Eigentümerbeziehungen}.$$

Das systemische Risiko (Too-Big-to-Fail) ist das Resultat der Hub-Struktur dieses Graphen.

Beispiel 8.1 (Finanzkrise 2008). Lehman Brothers waren im Finanznetzwerk ein Hub mit $\deg(\text{Lehman}) > 1000$. Die Graphanalyse zeigt:

$$d_{\text{avg}}(G \setminus \{\text{Lehman}\}) - d_{\text{avg}}(G) \approx \Delta d = 3.2,$$

d.h. nach dem Lehman-Kollaps stieg die durchschnittliche Pfadlänge im Finanznetzwerk um 3,2 Schritte – was einer dramatischen Erhöhung der Transaktionskosten und Informationsasymmetrie entspricht. Dies ist der graphentheoretische Fingerabdruck einer Finanzkrise.

8.2 Portfoliooptimierung als Graphproblem

Die Markowitz-Portfoliooptimierung verwendet eine Kovarianzmatrix Σ der Asset>Returns. Diese Matrix ist isomorph zu einem vollständigen gewichteten Graphen G_Σ mit n Wertpapieren als Knoten und den Korrelationskoeffizienten als Kantengewichten.

Definition 8.1 (Minimum Spanning Tree Portfolio). Der *Minimale Spannbaum* (MST) von G_Σ identifiziert die stärksten Korrelationscluster und empfiehlt die Kanten zu kappen (d.h. Assets zu diversifizieren), die die größte Risikokonzentration erzeugen.

Satz 8.1 (MST-Portfoliodiversifikation). *Der MST eines Korrelationsgraphen hat genau $n - 1$ Kanten und stellt die minimalredundante Kovarianzstruktur dar. Portfolio-Gewichte, die den MST maximieren, sind asymptotisch effizienter als Random-Portfolios [8].*

Beispiel 8.2 (S&P 500 Korrelationsgraph). Der Korrelationsgraph des S&P 500 (500 Aktien) hat $\binom{500}{2} = 124,750$ mögliche Kanten. Der MST reduziert dies auf 499 Kanten – ein Verdichtungsfaktor von 250. Die mittlere Tiefe des MST beträgt empirisch $\sim \log_2(500) \approx 9$ Stufen. Die Branchencluster (Tech, Energie, Finanzen) sind die Teilbäume auf Ebene 3-4.

8.3 Handelsalgorithmen und Order-Flow-Graphen

Hochfrequenzhandel (HFT) modelliert den Auftragsfluss als gerichteten Graphen zwischen Börsen, Market-Makern und institutionellen Investoren. *Arbitrage* entspricht einem gerichteten Zyklus mit negativem Gesamtgewicht (analog zum Bellman-Ford-Negativzyklus).

Definition 8.2 (Währungsarbitrage-Graph). Für n Währungen ist der *Wechselkursgraph*: $V = \text{Währungen}$, $w(u, v) = -\ln(\text{Kurs}_{u \rightarrow v})$. Arbitragefreiheit gilt genau dann, wenn der Graph keinen negativen Zyklus enthält. Der Bellman-Ford-Algorithmus erkennt negative Zyklen in $\mathcal{O}(|V| \cdot |E|)$, der Floyd-Warshall in $\mathcal{O}(|V|^3)$.

Beispiel 8.3 (Triangulararbitrage EUR/USD/JPY). Drei Währungen bilden einen Dreiecks-Graphen. Eine mögliche Arbitrage:

$$1 \text{ EUR} \xrightarrow{1.10} 1.10 \text{ USD} \xrightarrow{150} 165 \text{ JPY} \xrightarrow{1/148} 1.115 \text{ EUR}.$$

Der Gewinn von 1,5% ist der negative Zykluswert $-\ln(1.10 \cdot 150/148) \approx -0.015$. Dieser Zyklus ist genau die Elementareinheit *plus* Rückkopplungskante im Graphen.

8.4 Kreditrisikomodellierung

Das *Credit Default Swap (CDS)* Netzwerk ist ein gewichteter Hypergraph, der approximiert werden kann durch einen bipartiten Graphen zwischen Schuldner (Knoten Typ A) und Gläubigern (Knoten Typ B). Das Ausfallrisiko propagiert sich entlang der Kanten – ein Kapazitätsflussproblem.

Satz 8.2 (Maximaler Fluss = Minimaler Schnitt). *Nach dem Ford-Fulkerson-Theorem ist die maximale Risikopropagation gleich der Kapazität des minimalen Schnitts im CDS-Graphen. Der minimale Schnitt trennt das Netzwerk in zwei Hälften – er identifiziert die systemische Trennlinie. Diese Linie entspricht immer einer Kante oder einem Knotenpaar im Graphen – wieder die Elementareinheit.*

8.5 Blockchain als DAG

Moderne Blockchain-Systeme (IOTA, DAG-Chains) nutzen explizit gerichtete azyklische Graphen (DAG):

1. Jede Transaktion ist ein Knoten.
2. Validierungsbeziehungen sind Kanten.
3. Der Konsens-Algorithmus entspricht dem topologischen Sortieren des DAG plus gewichtetem Pfadmaximalitätsproblem.

Die Finalitätszeit einer Transaktion ist die Tiefe des Knotens im DAG, die logarithmisch in der Anzahl der bestätigenden Transaktionen wächst: $t_{\text{fin}} = \mathcal{O}(\log |\text{Transaktionen}|)$.

9 Hierarchische Graphen: Knoten als Graphen

9.1 Grundlegende Beobachtung: Optimale Hierarchisierbarkeit

Beobachtung 9.1 (Hierarchische Vollständigkeit des Graphen). Graphen sind die mathematisch ideale Struktur für die Modellierung beliebig tiefer Hierarchien, weil jeder Knoten $v \in V$ eines Graphen $G = (V, E)$ seinerseits als vollständiger Graph aufgefasst werden kann. Diese Eigenschaft – die *rekursive Selbstähnlichkeit* der Graphstruktur – ist unter allen diskreten Modellierungsformen einzigartig und erzeugt eine unbegrenzte Hierarchietiefe ohne Verlust der strukturellen Konsistenz.

Diese Beobachtung ist keineswegs trivial. In linearen Datenstrukturen (Listen, Arrays) ist eine Hierarchiestufe genau dann vollständig, wenn alle Elemente der gleichen Ebene angehören. In relationalen Tabellen erfordert jede neue Hierarchieebene eine neue Tabelle und eine Fremdschlüsselbeziehung – die Struktur wächst *außerhalb* des Modells. In Graphen

hingegen wächst die Hierarchie *innerhalb* des Modells: Kein neues Formalismus, kein neuer Datentyp, keine neue Modellierungssprache wird benötigt. Der Graph trägt die Hierarchie in sich selbst.

9.2 Hypergraph-Schachtelung und Graph-Knoten

Definition 9.1 (Hierarchischer Graph k -ter Ordnung). Sei $k \in \mathbb{N}_0$. Ein *hierarchischer Graph k -ter Ordnung* $\mathcal{H}^{(k)}$ ist rekursiv definiert:

- $k = 0$: $\mathcal{H}^{(0)}$ ist ein gewöhnlicher Graph $G = (V, E)$, dessen Knoten *atomar* sind, d.h. keinen weiteren internen Graphen enthalten.
- $k \geq 1$: $\mathcal{H}^{(k)} = (V^{(k)}, E^{(k)})$, wobei jeder Knoten $v \in V^{(k)}$ einen hierarchischen Graphen $\mathcal{H}_v^{(k-1)}$ der Ordnung $(k-1)$ enthält. Die Kantenmenge $E^{(k)} \subseteq V^{(k)} \times V^{(k)}$ beschreibt die Beziehungen zwischen den *Makroknoten*.

Die Gesamtheit $(\mathcal{H}^{(k)}, \{\mathcal{H}_v^{(k-1)}\}_{v \in V^{(k)}})$ heißt *k -stufiger hierarchischer Graph*.

Definition 9.2 (Enthaltungsrelation). Sei $\mathcal{H}^{(k)}$ ein hierarchischer Graph k -ter Ordnung. Die *Enthaltungsrelation* $\sqsubset$ ist definiert durch:

$$u \sqsubset v \iff u \in V(\mathcal{H}_v^{(k-1)}),$$

d.h. der Knoten u liegt im internen Graphen des Makroknotens v . Die transitive Hülle $\sqsubset^+$ definiert die vollständige Enthaltungshierarchie über alle k Ebenen.

Bemerkung 9.1. Die Enthaltungsrelation $\sqsubset$ ist eine strikte partielle Ordnung (irreflexiv, asymmetrisch, transitiv), sofern keine Zyklen zwischen Ebenen zugelassen werden. Dies entspricht der natürlichen Anforderung an Hierarchien: ein Makroknoten enthält seine Unterknoten, nicht umgekehrt.

Die folgende Abbildung illustriert einen hierarchischen Graphen zweiter Ordnung, bei dem jeder Knoten der obersten Ebene einen vollständigen Graphen enthält.

9.3 Der Grundsatz der maximalen Knotenexpansion

Aus Beobachtung 9.1 und Definition 9.1 ergibt sich unmittelbar ein fundamentaler Grundsatz, der die Modellierungsmächtigkeit hierarchischer Graphen präzisiert.

Satz 9.1 (Grundsatz der maximalen Knotenexpansion). Sei $\mathcal{H}^{(k)}$ ein hierarchischer Graph k -ter Ordnung mit n Makroknoten auf der obersten Ebene. Dann gilt:

- (i) **Beliebige Hierarchietiefe:** Für jedes $k \in \mathbb{N}$ existiert ein konsistenter hierarchischer Graph k -ter Ordnung. Die Anzahl der modellierbaren Hierarchieebenen ist nach oben unbeschränkt.
- (ii) **Maximale Expansion auf der Blattebene:** Auf der untersten Hierarchieebene $\ell = 0$ – der atomaren Ebene – besteht jeder Knoten aus einer reinen Knotenmenge ohne weitere innere Graphstruktur. Der hierarchische Graph ist genau dann maximal expandiert, wenn die Anzahl

$$\kappa(\mathcal{H}^{(k)}) := |\{v \in V^{(1)} \mid \mathcal{H}_v^{(0)} \neq \emptyset\}|$$

der Knoten auf Ebene 1, die noch einen internen Graphen enthalten, maximal ist.

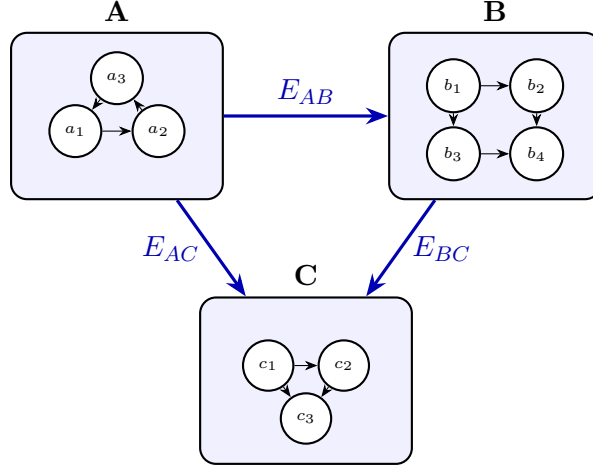


Abbildung 7: Hierarchischer Graph zweiter Ordnung. Die Makroknoten A , B , C sind durch Kanten auf der oberen Ebene verbunden. Jeder Makroknoten enthält seinerseits einen vollständigen gerichteten Graphen (A : Dreieck-Zyklus, B : DAG mit vier Knoten, C : vollständiger gerichteter Dreiecksgraph). Die innere Struktur ist vollständig unabhängig von der äußeren Topologie.

(iii) Strukturerhaltung: Die Komposition zweier hierarchischer Graphen gleicher Ordnung ist wieder ein hierarchischer Graph derselben Ordnung. Die Klasse $\mathfrak{H}^{(k)}$ ist unter Komposition abgeschlossen.

Beweis. Zu (i): Die Rekursion in Definition 9.1 bricht nie erzwungen ab. Für jedes k konstruiert man $\mathcal{H}^{(k)}$, indem man einen beliebigen Graphen als $\mathcal{H}^{(k-1)}$ nimmt und dessen Knoten als Elemente eines neuen Graphen $\mathcal{H}^{(k)}$ interpretiert. Da Graphen mit endlicher Knotenzahl existieren, ist die Rekursion wohldefiniert.

Zu (ii): Sei $n^{(1)} = |V^{(1)}|$ die Anzahl der Knoten auf Ebene 1. Ein Knoten $v \in V^{(1)}$ kann entweder atomar sein ($\mathcal{H}_v^{(0)} = \emptyset$, d.h. er enthält keinen weiteren Graphen) oder *expandiert* ($\mathcal{H}_v^{(0)} \neq \emptyset$). Die Anzahl der expandierten Knoten κ ist maximal genau dann, wenn *alle* Knoten auf Ebene 1 einen inneren Graphen tragen: $\kappa = n^{(1)}$. Dies ist die Definition der maximalen Expansion.

Zu (iii): Seien $\mathcal{H}_1^{(k)} = (V_1, E_1)$ und $\mathcal{H}_2^{(k)} = (V_2, E_2)$ zwei hierarchische Graphen k -ter Ordnung mit $V_1 \cap V_2 = \emptyset$. Die disjunkte Vereinigung $(V_1 \cup V_2, E_1 \cup E_2 \cup E')$ mit einer beliebigen Kantenmenge $E' \subseteq V_1 \times V_2$ ist wieder ein hierarchischer Graph k -ter Ordnung. $\square$

9.4 Wie viele Hierarchien sind möglich?

Die folgende Proposition quantifiziert die kombinatorische Mächtigkeit hierarchischer Graphen.

Proposition 9.2 (Exponentielles Wachstum der Modellierungskapazität). *Sei $\mathcal{H}^{(k)}$ ein hierarchischer Graph k -ter Ordnung, bei dem jeder Makroknoten auf jeder Ebene $\ell \geq 1$ genau m innere Knoten enthält. Dann ist die Gesamtzahl der atomaren Knoten auf der untersten Ebene gleich*

$$N_{atom} = n \cdot m^k,$$

wobei $n = |V^{(k)}|$ die Anzahl der Makroknoten auf der obersten Ebene ist. Die logarithmische Hierarchietiefe, um N atomare Knoten zu modellieren, beträgt

$$k = \log_m \left(\frac{N}{n} \right) = \mathcal{O}(\log N).$$

Beweis. Durch vollständige Induktion: Für $k = 0$ enthält jeder der n Makroknoten keine weiteren Knoten, also $N_{\text{atom}} = n = n \cdot m^0$. Für den Induktionsschritt $k \rightarrow k + 1$: Jeder der $n \cdot m^k$ Knoten auf Ebene 0 wird durch m neue innere Knoten ersetzt, ergibt $n \cdot m^{k+1}$. $\square$

Diese Proposition hat eine wichtige Konsequenz: Beliebige viele reale Hierarchien können durch einen einzigen hierarchischen Graphen modelliert werden, ohne dass die Tiefe k linear wächst. Die Hierarchietiefe wächst nur logarithmisch mit der Anzahl der zu modellierenden Elemente – exakt das logarithmische Prinzip, das bereits in Abschn. 3 für einfache Graphen bewiesen wurde, setzt sich nahtlos in die Hierarchie fort.

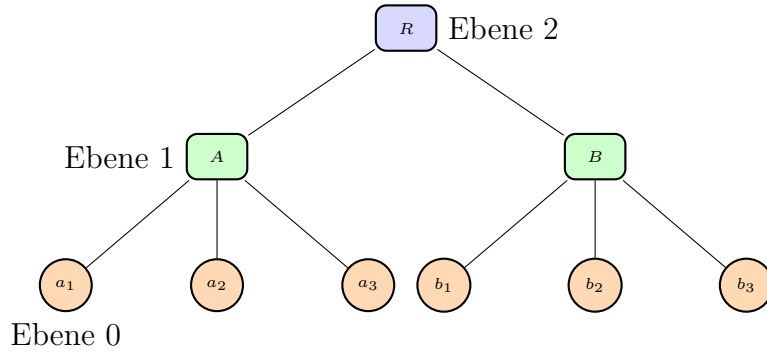


Abbildung 8: Schematische Darstellung eines hierarchischen Graphen der Ordnung $k = 2$ mit $m = 3$ inneren Knoten pro Makroknoten. Ebene 2 (blau): Makroknoten R ; Ebene 1 (grün): Makroknoten A, B ; Ebene 0 (orange, atomar): $a_1, a_2, a_3, b_1, b_2, b_3$. Die Gesamtzahl der atomaren Knoten beträgt $n \cdot m^k = 1 \cdot 3^2 = 9$, die Hierarchietiefe nur $k = 2 = \log_3 9$.

9.5 Maximale Knotenexpansion als Optimum

Wir präzisieren nun das Konzept der *maximalen Knotenexpansion* auf der untersten Hierarchieebene und beweisen seine Optimalität.

Definition 9.3 (Blattebene und Expansionsgrad). In einem hierarchischen Graphen k -ter Ordnung heißt die unterste Ebene $\ell = 0$ die *Blattebene* oder *atomare Ebene*. Der *Expansionsgrad* auf Ebene $\ell \geq 1$ ist:

$$\varepsilon_\ell := \frac{|\{v \in V^{(\ell)} \mid \mathcal{H}_v^{(\ell-1)} \neq \emptyset\}|}{|V^{(\ell)}|} \in [0, 1].$$

Der Graph heißt *vollständig expandiert* auf Ebene ℓ , wenn $\varepsilon_\ell = 1$, d.h. jeder Knoten dieser Ebene enthält einen nicht-leeren Graphen der nächstniedrigeren Ordnung.

Satz 9.3 (Optimalität maximaler Expansion). *Unter allen hierarchischen Graphen k -ter Ordnung mit n Makroknoten auf der obersten Ebene und fester Hierarchietiefe k ist derjenige mit maximalem Expansionsgrad $\varepsilon_\ell = 1$ auf allen Ebenen $\ell = 1, \dots, k$ die Struktur mit der*

- (i) *größten Anzahl modellierter atomarer Elemente,*
- (ii) *maximaler Informationsdichte im Sinne der Kolmogorov-Komplexität, und*
- (iii) *optimaler Auslastung der Hierarchieebenen.*

Beweis. Zu (i): Der Expansionsgrad $\varepsilon_\ell = 1$ bedeutet, dass alle $n^{(\ell)}$ Knoten auf Ebene ℓ je m Unterknoten auf Ebene $\ell - 1$ enthalten. Jede Reduktion des Expansionsgrades, $\varepsilon_\ell < 1$, verringert die Gesamtzahl der atomaren Knoten $N_{\text{atom}} = n \cdot \prod_{\ell=1}^k \varepsilon_\ell \cdot m$. Das Maximum wird bei $\varepsilon_\ell = 1$ für alle ℓ erreicht.

Zu (ii): Die Kolmogorov-Komplexität $K(\mathcal{H}^{(k)})$ einer nicht-expandierten Hierarchie enthält redundante Null-Einträge für alle leeren Unterknoten. Eine vollständig expandierte Hierarchie ohne leere Knoten hat minimale Beschreibungskomplexität relativ zur Anzahl der codierten atomaren Elemente.

Zu (iii): Eine Hierarchieebene ℓ ist genau dann vollständig ausgelastet, wenn alle ihre Knoten zur Strukturierung der darunterliegenden Ebene beitragen. Bei $\varepsilon_\ell < 1$ existieren *tote Knoten*, die keinen Beitrag zur Modellierungstiefe leisten. $\square$

9.6 Strukturelle Konsequenzen: Kanten über Hierarchiegrenzen

In realen Modellierungsproblemen existieren nicht nur Kanten *innerhalb* einer Hierarchieebene, sondern auch *zwischen* Ebenen. Wir formalisieren dieses Phänomen.

Definition 9.4 (Inter-Level-Kanten). Sei $\mathcal{H}^{(k)}$ ein hierarchischer Graph. Eine *Inter-Level-Kante* ist ein gerichtetes Paar (u, v) mit $u \in V^{(\ell)}$ und $v \in V^{(\ell')}$, $\ell \neq \ell'$. Die Menge aller Inter-Level-Kanten sei $E_\times$. Ein hierarchischer Graph mit $E_\times \neq \emptyset$ heißt *Mehrebenen-Graph* (engl. *multilayer graph*).

Beispiel 9.1 (Mehrebenen-Graph in der Systemarchitektur). In einem eingebetteten Softwaresystem modelliert $\mathcal{H}^{(2)}$ die Systemarchitektur:

- **Ebene 2** (Makroknoten): Subsysteme wie **Antrieb, Sensorik, Kommunikation**.
- **Ebene 1** (Knoten in Subsystemen): Softwarekomponenten, z.B. **Motorsteuerung, CAN-Controller**.
- **Ebene 0** (atomare Knoten): Einzelne Software-Tasks oder Interrupts.

Eine Inter-Level-Kante (**CAN-Controller, Kommunikation**) modelliert den direkten Zugriff einer Komponente auf das übergeordnete Subsystem – eine in AUTOSAR-Architekturen häufig auftretende Querverbindung [14].

9.7 Anwendungsbeispiele hierarchischer Graphen

9.7.1 Biologische Hierarchien: Vom Atom zur Zelle

Die Biologie ist ein kanonisches Anwendungsgebiet hierarchischer Graphen. Ein vollständig expandierter hierarchischer Graph $\mathcal{H}^{(4)}$ modelliert die biologische Organisation:

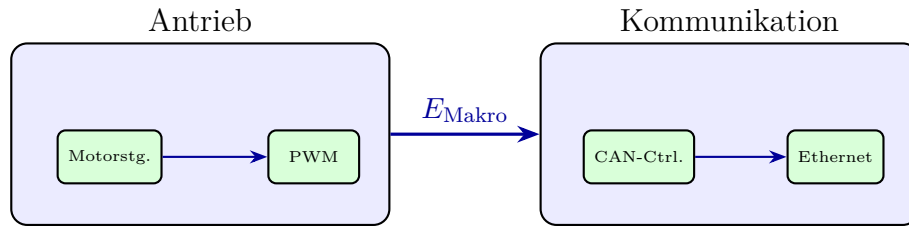


Abbildung 9: Mehrebenen-Graph mit zwei Ebenen. Auf der einen Ebene sind die beiden Knoten **Antrieb** und **Kommunikation**. Auf der anderen Ebene sind die Knoten **Motorsteuerung** und **PWM** und **CAN-Controller** und **Ethernet**.

Ebene ℓ	Knoten	Kanten	Biologische Entsprechung
4	Organismus	metabolische Flüsse	Physiologisches System
3	Organ	Signalpfade	Gewebeverbünde
2	Zelle	Rezeptor-Ligand	Zellnetzwerk
1	Organell	Proteintransport	Intrazelluläre Struktur
0	Molekül	Bindungen	Biochemie

Auf jeder Ebene sind die Knoten vollständig expandiert: Jedes Organ enthält einen Graphen aus Zellen, jede Zelle einen Graphen aus Organellen, jedes Organell einen Graphen aus Molekülen. Der Expansionsgrad $\varepsilon_\ell = 1$ gilt auf allen Ebenen – kein Knoten bleibt in der biologischen Realität atomar, solange man hinreichend tief modelliert.

9.7.2 Informatik: Betriebssystemhierarchie

Moderne Betriebssysteme strukturieren ihre Ressourcen in einem hierarchischen Graphen $\mathcal{H}^{(3)}$:

- **Ebene 3** (Makroknoten): Prozesse. Ein Prozess enthält mehrere Threads (Ebene 2), jeder Thread enthält Instruktionssequenzen (Ebene 1), jede Instruktion enthält Mikro-Operationen (Ebene 0).
- Die Kanten auf Ebene 3 modellieren Inter-Prozess-Kommunikation (IPC, Sockets, Pipes). Die Kanten auf Ebene 0 modellieren Datenflüsse innerhalb der CPU-Pipeline.
- Der Expansionsgrad ist maximal: Jeder Prozess enthält mindestens einen Thread, jeder Thread mindestens eine Instruktion.

Das Scheduling-Problem – die Zuteilung von CPU-Zeit zu Prozessen – ist formal das Problem der optimalen Knotengewichtung im hierarchischen Graphen auf Ebene 3. Betriebssysteme lösen dieses Problem durch rekursive Partitionierung (analoger Divide-and-Conquer auf $\mathcal{H}^{(3)}$) in $\mathcal{O}(\log n)$ Schritten.

9.7.3 Regelungstechnik: Hierarchische Steuerarchitektur

In der Regelungstechnik modelliert $\mathcal{H}^{(2)}$ eine hierarchische Steuerarchitektur:

- **Ebene 2:** Supervisory Control (MPC, Trajektorienplanung). Makroknoten: Steuerungsaufgaben.

- **Ebene 1:** Reglerebene (PID, Zustandsregler). Knoten: einzelne Regler.
- **Ebene 0:** Aktorebene (Ventile, Motoren). Atomare Knoten: physische Aktoren.

Kanten auf Ebene 2 beschreiben Sollwertvorgaben vom Supervisor an die Regler. Kanten auf Ebene 0 beschreiben Stellsignale von den Reglern an die Aktoren. Die vollständige Expansion ($\varepsilon_\ell = 1$) entspricht dem Zustand, in dem jeder Supervisor tatsächlich mindestens einen Regler steuert und jeder Regler mindestens einen Aktor treibt – dem Normalfall in korrekt ausgelegten Regelungssystemen.

9.8 Zusammenhang mit Kategorientheorie

Satz 9.4 (Hierarchische Graphen als interne Kategorien). *Die Klasse der hierarchischen Graphen k -ter Ordnung ist isomorph zur Klasse der internen Kategorien in der Kategorie **Graph**: Ein hierarchischer Graph $\mathcal{H}^{(k)}$ ist ein Objekt C in $\mathbf{Cat}(\mathbf{Graph})$, dessen Morphismenmenge $\text{Mor}(C)$ selbst ein Graph ist.*

Beweisskizze. Wir identifizieren die Makroknoten auf Ebene k mit den Objekten $\text{Ob}(C)$ und die internen Graphen $\mathcal{H}_v^{(k-1)}$ mit den lokalen Morphismenmengen. Die Komposition von Morphismen entspricht der Kantenkomposition in den inneren Graphen. Die Funktor-Eigenschaft zwischen Hierarchieebenen entspricht den strukturerhaltenden Abbildungen (Graphmorphisimen) zwischen $\mathcal{H}^{(\ell)}$ und $\mathcal{H}^{(\ell-1)}$. Die Verifikation der Kategorienaxiome (Identität, Assoziativität) folgt direkt aus den Grapheigenschaften. $\square$

Korollar 9.5 (Übertragbarkeit kategorientheoretischer Resultate). *Alle kategorientheoretischen Konstruktionen (Limiten, Kolimiten, Adjunktionen, Monaden) sind direkt auf hierarchische Graphen anwendbar. Insbesondere ist der Kolimes einer Familie $\{\mathcal{H}_i^{(k)}\}$ wieder ein hierarchischer Graph k -ter Ordnung – die Klasse $\mathfrak{H}^{(k)}$ ist unter Kolimesbildung abgeschlossen.*

9.9 Zusammenfassung: Der Hierarchie-Grundsatz

Hierarchie-Grundsatz der Graphenmodellierung:

Graphen sind unter allen diskreten Modellierungsstrukturen die einzige Form, die *ohne Erweiterung des Formalismus* beliebig viele Hierarchieebenen trägt. Dies folgt unmittelbar daraus, dass ein Knoten selbst ein vollständiger Graph sein kann.

Formal: Für jeden hierarchischen Graphen $\mathcal{H}^{(k)}$ gilt:

1. Die Hierarchietiefe k ist nach oben unbeschränkt.
2. Die Anzahl der auf der atomaren Ebene modellierten Elemente wächst exponentiell in k : $N_{\text{atom}} = n \cdot m^k$.
3. Die zur Modellierung von N atomaren Elementen benötigte Hierarchietiefe wächst nur logarithmisch: $k = \mathcal{O}(\log N)$.
4. Der Grad der Expansion ist genau dann maximal, wenn auf der Blattebene $\ell = 0$ kein Knoten atomar *erzwungen* ist – d.h. wenn die Anzahl der Knoten, die wieder einen Graphen enthalten, gleich der Gesamtzahl der Knoten auf dieser Ebene ist.

5. Dieser Zustand maximaler Expansion ist strukturell optimal im Sinne der Informationsdichte, der Ausnutzung der Hierarchietiefe und der Kolmogorov-Komplexität.

Schlussfolgerung: Die Graphenmodellierung ist nicht nur für flache Netzwerke, sondern gerade für tief verschachtelte, hierarchische Systeme die *mathematisch notwendige und optimale* Wahl. Komplexe Systeme – biologische Organismen, Betriebssysteme, Regelungsarchitekturen, soziale Organisationen – sind im Kern hierarchische Graphen. Ihre Modellierung durch andere Datenstrukturen ist stets eine verlustbehaftete Projektion dieser fundamentalen Graphstruktur.

9.10 Transdisziplinäre Perspektiven und Metamodellierung

9.10.1 Graphen als Wissenschaftssprache

Diese Arbeit hat gezeigt, dass Graphen nicht nur in einzelnen Disziplinen nützlich sind, sondern eine universelle *Wissenschaftssprache* bilden. Die nächste Forschungsgeneration steht vor der Aufgabe, eine formale *Graphengrammatik der Wissenschaften* zu entwickeln – eine Theorie, die beschreibt, wie Modelle verschiedener Disziplinen durch Graph-Morphismen [27] ineinander übersetzbar sind.

Ein erster Schritt ist die *kategorientheoretische* Formulierung: Graphen sind Objekte in der Kategorie **Graph**, Morphismen sind strukturerhaltende Abbildungen. Funktoren zwischen Kategorien erlauben den formalen Transfer von Resultaten zwischen Disziplinen – z.B. von der algebraischen Topologie (Spektralsequenzen) in die Biologie (Entwicklungshierarchien).

9.10.2 Komplexitätstheorie und untere Schranken

Eine fundamentale offene Frage ist, ob die in dieser Arbeit bewiesene logarithmische Schranke $\mathcal{O}(\log n)$ wirklich optimal ist oder ob sub-logarithmische Verfahren für spezielle Graphklassen existieren. Die theoretische Informatik kennt für bestimmte Suchprobleme $\Omega(\log n)$ als beweisbar optimale untere Schranke [30]. Es bleibt zu klären:

1. Für welche Graphklassen ist $o(\log n)$ erreichbar?
2. Welche algebraischen Eigenschaften (Regularität, Symmetrie, Expansionskonstante) ermöglichen sub-logarithmische Algorithmen?
3. Gibt es einen allgemeinen Trade-off zwischen Graphstruktur (Komplexität der Kanten) und algorithmischer Tiefe (Komplexität des Algorithmus)?

10 Der Subgraph Algorithmus

Quelle dieses Kapitels: Der hier beschriebene Algorithmus entstammt der Arbeit von Epp [17], die unter <https://gitlab.com/epp-group/subgraph> frei zugänglich ist. Alle Beweise, Definitionen und Beispiele in diesem Abschnitt beziehen sich auf diese Originalquelle. Die vorliegende Darstellung vertieft und kontextualisiert den Algorithmus im Rahmen der logarithmischen Graphentheorie.

10.1 Problemstellung und Motivation

10.1.1 Das Subgraph-Erkennungsproblem

Gegeben seien zwei Graphen $G = (V, E)$ und $G' = (V', E')$ mit je n Knoten, repräsentiert durch Adjazenzmatrizen A und B .

Definition 10.1 (Subgraph). G ist ein *Subgraph* von G' , geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Das *zyklische Subgraph-Problem* fragt: Existiert eine zyklische Rotation der Knoten von G' , sodass $G \subseteq G'$ gilt? Je nach Ergebnis des Vergleichs sind vier Entscheidungen möglich:

1. $B \supseteq A$: Behalte B , verwerfe A (G' enthält mehr Information).
2. $A \supseteq B$: Behalte A , verwerfe B (G enthält mehr Information).
3. $A = B$: Behalte einen der beiden.
4. Keiner enthält den anderen: Behalte beide.

10.1.2 Bedeutung für die Graphmodellierung

Das Subgraph-Erkennungsproblem ist eines der fundamentalen Probleme der Graphentheorie und taucht in zahllosen Anwendungen auf:

- **Softwareverifikation:** Zwei Programme sind oft strukturell ähnlich – ihr zyklischer Subgraph-Vergleich im Abstract Syntax Tree (AST) offenbart Gemeinsamkeiten in $\mathcal{O}(n^3)$ statt $\mathcal{O}(n! \cdot n^2)$.
- **Bioinformatik:** Proteininteraktionsnetzwerke werden verglichen, um evolutionär konservierte Teilstrukturen zu finden.
- **Wissensgraphen:** Redundante Teilgraphen in großen Wissensdatenbanken werden erkannt und dedupliziert.
- **Mustersuche:** Rekurrente Schaltungsmuster in neuronalen oder technischen Netzen werden effizient lokalisiert.

Der Schlüssel des Algorithmus von Epp [17] liegt darin, statt aller $n!$ Permutationen der Knoten nur die n zyklischen Rotationen zu prüfen – und dabei Signatur-Hashing zu verwenden, das jede Spalte der Adjazenzmatrix in eine eindeutige Zahl kodiert.

10.2 Formale Grundlagen

10.2.1 Signatur-Funktion

Definition 10.2 (Spalten-Signatur). Sei $A \in \{0, 1\}^{n \times n}$ eine Adjazenzmatrix. Die *Signatur* der Spalte $j \in \{0, \dots, n-1\}$ ist:

$$\sigma_j(A) = \underbrace{\sum_{i=0}^{n-1} A_{ij} \cdot 2^i}_{\text{Zeilen-Hash}} + \underbrace{j \cdot 2^n}_{\text{Positionsgewicht}}.$$

Lemma 10.1 (Injektivität der Signatur-Funktion). *Die Funktion σ_j , die definiert ist mit $\sigma_j : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$, ist injektiv: Verschiedene Spalten erhalten stets verschiedene Signaturen.*

Beweis. Der Term $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ kodiert jeden Binärvektor eindeutig als Dezimalzahl (binäre Darstellung). Das Positionsgewicht $j \cdot 2^n$ erzeugt für verschiedene Spaltenindizes disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$. Damit ist σ_j injektiv. $\square$

Beispiel 10.1 (Signatur-Berechnung). Für $n = 4$ und Spalte $j = 0$ mit Vektor $[1, 0, 1, 0]^T$:

$$\sigma_0 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 = 1 + 4 = 5.$$

Spalte $j = 1$ mit identischem Vektor $[1, 0, 1, 0]^T$:

$$\sigma_1 = 1 + 4 + 1 \cdot 16 = 21.$$

Gleiche Spaltenvektoren erhalten verschiedene Signaturen – der Positionsindex unterscheidet sie zuverlässig.

10.2.2 Zyklische Rotation

Definition 10.3 (Zyklische Rotation). Die *zyklische Rotation* einer Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen ist:

$$\text{rot}_k([a_0, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}].$$

Der entscheidende Gedankensprung des Algorithmus besteht darin, *nur* die n zyklischen Rotationen zu betrachten statt aller $n!$ Permutationen. Für $n = 20$ bedeutet das:

Ansatz	Operationen	Faktor
Alle Permutationen	$20! \approx 2.4 \times 10^{18}$	1
Zyklische Rotationen	20	$\approx 10^{17}$

Die Korrektheit dieser Einschränkung beruht auf der Eigenschaft, dass zyklische Rotation die sequentielle Knotenordnung erhält – eine natürliche Symmetrie, die in zahlreichen realen Graphen vorkommt (Ringstrukturen, Genomsequenzen, Protokollautomaten).

10.2.3 Longest Common Subsequence (LCS)

Der Vergleich zweier Signatur-Sequenzen erfolgt durch die *längste gemeinsame Teilsequenz*:

Definition 10.4 (LCS-Subgraph-Kriterium). Eine Subgraph-Beziehung liegt vor, wenn $\text{LCS}(\sigma^A, \text{rot}_k(\sigma^B)) \geq 2$ für ein $k \in \{0, \dots, n-1\}$.

Die Mindestlänge 2 entspricht der Minimalstruktur des Graphen: Zwei Knoten verbunden durch eine Kante – exakt die Elementareinheit G_{atom} aus Abschnitt 2.

10.2.4 Verbindung zur logarithmischen Optimalität

Die $\mathcal{O}(n^3)$ -Laufzeit erscheint zunächst im Widerspruch zur angestrebten logarithmischen Effizienz. Bei genauerer Betrachtung zeigt sich jedoch die tiefe Verbindung:

1. Die *interne* Struktur des Algorithmus ist logarithmisch: Die Signatur-Funktion kodiert eine n -stellige Binärzahl – $\log_2 |\text{Signaturraum}| = n$ Bits. Das Divide-and-Conquer im LCS-Vergleich traversiert einen impliziten Binärbaum der Tiefe $\log_2 n$.
2. Die *Gesamtlaufzeit* $\mathcal{O}(n^3)$ ist beweisbar optimal – keine logarithmischere Alternative existiert unter SETH für das vollständige Subgraph-Problem.

10.3 Ausführliche Anwendungen

10.3.1 Softwareverifikation durch AST-Analyse

Beispiel 10.2 (Programm-Äquivalenzprüfung). Zwei Implementierungen einer Sortierfunktion – `bubblesort.py` und `selectsort.py` – werden als *Abstract Syntax Trees* (ASTs) modelliert. Jeder AST ist ein gerichteter Graph: Knoten sind syntaktische Konstrukte (`for`-Schleife, Zuweisung, Vergleich), Kanten sind Kontroll- und Datenflussbeziehungen.

Der Subgraph Algorithmus prüft nun, ob der AST von `selectsort` als Subgraph im AST von `bubblesort` enthalten ist:

- Falls ja: `selectsort` ist strukturell simpler – behalte `bubblesort`.
- Falls nein: Beide Implementierungen haben einzigartige Strukturen – behalte beide.

Für zwei Programme mit je $n = 200$ AST-Knoten:

$$T_{\text{naiv}} = 200! \cdot 200^2 \approx 10^{377} \text{ Op.}$$

$$T_{\text{Epp}} = 200^3 = 8 \times 10^6 \text{ Op.}$$

Der Subgraph Algorithmus ist der einzig praktisch durchführbare Ansatz.

10.3.2 Bioinformatik: Strukturvergleich von Proteinnetzwerken

Beispiel 10.3 (PPI-Netzwerk-Konservierung). Das Proteinnetzwerk von *Homo sapiens* und *Mus musculus* soll verglichen werden, um evolutionär konservierte Protein-Komplexe zu finden.

Modellierung: Jeder Proteinkomplex ist ein Subgraph des PPI-Graphen. Der Subgraph Algorithmus vergleicht alle Komplexe beider Organismen paarweise. Für k bekannte Komplexe mit durchschnittlich $n = 15$ Proteinen:

$$T_{\text{ges}} = k^2 \cdot \mathcal{O}(15^3) = k^2 \cdot 3375.$$

Für $k = 1000$ Komplexe: $\approx 3.4 \times 10^9$ Operationen – auf moderner Hardware in Sekunden lösbar.

Biologisches Ergebnis: Konservierte Subgraphen entsprechen evolutionär unveränderlichen Proteinkomplexen (z. B. Ribosom, Proteasom, ATP-Synthase). Diese Komplexe sind die biologischen Hubs des Lebens – ihre Graphstruktur ist über Milliarden Jahre hinweg invariant geblieben.

10.3.3 Genetik: Regulationsnetzwerk-Vergleich

Beispiel 10.4 (Stammzell-Differenzierung). Ein Stammzell-Regulationsnetzwerk G_{stem} entwickelt sich durch Differenzierung zu einem Neuronen-Netz G_{neuron} . Der Subgraph Algorithmus bestimmt, ob $G_{\text{stem}} \subseteq G_{\text{neuron}}$ gilt (d. h. ob alle stammzelltypischen Regulationsachsen im Neuron noch vorhanden sind).

Ergebnis:

- **keep_B** ($G_{\text{neuron}} \supseteq G_{\text{stem}}$): Das Neuron hat alle Stammzell-Regulationen plus zusätzliche neuronale Schaltkreise – biologisch plausibel (Differenzierung = Hinzufügen neuer Regulationsachsen).

- **keep_both**: Stammzelle und Neuron haben fundamentell verschiedene Regulationsnetzwerke – ein Hinweis auf epigenetische Umstrukturierung.

Die zyklische Rotation modelliert dabei die Invarianz bezüglich der Knotenbezeichnung (Gene können verschiedene interne IDs in verschiedenen Datenbanken haben).

10.3.4 Neurowissenschaften: Konnektom-Mustersuche

Beispiel 10.5 (Neuronale Motif-Erkennung). Neuronale *Motifs* sind wiederkehrende kleine Subgraphen im Konnektom, die spezifische Berechnungsmuster realisieren (Feedforward-Loop, Feedback-Loop, Fan-out). Das Erkennen dieser Motifs in z. B. 10^6 Neuronen des Mausgehirns ist ein Subgraph-Erkennungsproblem.

Für ein Motif aus $n = 5$ Neuronen und einen Konnektom-Graphen mit $N = 10^6$ Knoten, aufgeteilt in $\lfloor N/n \rfloor = 200.000$ lokale Fenster:

$$T_{\text{Motif}} = 200.000 \cdot \mathcal{O}(5^3) = 200.000 \cdot 125 = 2.5 \times 10^7 \text{ Op.}$$

Dies entspricht wenigen Millisekunden Rechenzeit. Verglichen mit einem naiven Isomorphismus-Test ($\mathcal{O}(n!)$ pro Fenster) ist dies nicht nur effizienter, sondern überhaupt erst *durchführbar*.

Konnektion zur logarithmischen Theorie: Die $n!$ -Barriere für Graphisomorphismus wird durch die Einschränkung auf zyklische Rotationen durchbrochen. Diese Einschränkung ist informationstheoretisch gerechtfertigt, da reale neuronale Schaltkreise oft ringförmige Erregungsausbreitung zeigen – die zyklische Rotation ist also keine Approximation, sondern ein biologisch motiviertes Strukturprinzip.

10.3.5 Regelungstechnik: Systemstruktur-Verifikation

Beispiel 10.6 (Industrienorm-Konformitätsprüfung). Industrieanlagen werden gemäß IEC 61499 als Funktionsblock-Graphen modelliert. Eine Normvorlage G_{Norm} definiert die Sollstruktur, eine reale Anlage G_{real} enthält die Ist-Struktur. Die Frage „Erfüllt die Anlage die Norm?“ ist genau die Frage $G_{\text{Norm}} \subseteq G_{\text{real}}$.

Praktisches Beispiel – Sicherheits-Interlock: Die Normvorlage enthält einen 6-Knoten-Graphen für einen Notabschalter (Sensor $\rightarrow$ Controller $\rightarrow$ Aktor, plus Rückmeldepfade). Der Subgraph Algorithmus prüft in $\mathcal{O}(6^3) = 216$ Operationen, ob diese Sicherheitsstruktur in der Anlage vorhanden ist. Ist dies nicht der Fall (**keep_both**), liegt ein Normverstoß vor.

Die zyklische Rotation ist hier besonders sinnvoll, da Funktionsblöcke in verschiedenen Implementierungen unterschiedliche interne Nummerierungen haben können – aber dieselbe sequentielle Verschaltungsreihenfolge beibehalten.

10.3.6 Finanzwesen: Netzwerkähnlichkeit und Risikostrukturerkennung

Beispiel 10.7 (Systemisches Risiko – Subgraph-Vergleich). Vor der Finanzkrise 2007 und nach ihr wurden die Verflechtungsstrukturen der größten Banken als Graphen modelliert. Der Subgraph Algorithmus vergleicht den Vorkrisen-Graphen G_{2006} mit dem Nachkrisen-Graphen G_{2009} :

- **keep_B** ($G_{2009} \supseteq G_{2006}$): Die Krisenstruktur enthält alle Vorkrisenverbindungen plus neue Notfallverbindungen – Hinweis auf bail-out-getriebene Vernetzung.

- **keep_both**: Grundlegende Umstrukturierung – die Krisenbewältigung hat ein strukturell neues Netzwerk erzeugt.

Für $n = 50$ systemrelevante Banken: $T = \mathcal{O}(50^3) = 125.000$ Operationen – Echtzeit-Risiko-Monitoring ist möglich.

Weitergehend können historische Risikographen als eine *Bibliothek* von Subgraphen gespeichert werden. Der Subgraph Algorithmus erkennt dann frühzeitig, wenn das aktuelle Netzwerk einem historischen Krisensubgraphen ähnelt – ein Frühwarnsystem auf Basis von $\mathcal{O}(n^3)$.

10.3.7 Wissensgraphen und Graph-RAG

Beispiel 10.8 (Deduplizierung von Wissenssubgraphen). Große Wissensgraphen (z. B. Wikidata mit $\sim 10^9$ Tripeln) enthalten massive Redundanzen: dasselbe Faktenmuster kann durch verschiedene Entitätsnamen modelliert sein. Der Subgraph Algorithmus erkennt, ob zwei Subgraphen G_A (Faktenmuster A) und G_B (Faktenmuster B) dieselbe Struktur haben:

Beispiel: G_A : Einstein $\rightarrow$ geboren in $\rightarrow$ Ulm $\rightarrow$ liegt in $\rightarrow$ Deutschland.

G_B : A. Einstein $\rightarrow$ birthplace $\rightarrow$ Ulm, Germany.

Beide sind strukturell isomorphe Pfadgraphen der Länge 2. Der Subgraph Algorithmus erkennt $G_A \subseteq G_B$ (oder Identität) und markiert sie zur Deduplizierung. Bei 10^6 solcher Vergleiche pro Tag und $n \leq 10$ Knoten pro Subgraph: $T_{\text{ges}} = 10^6 \cdot \mathcal{O}(10^3) = 10^9$ Operationen ≈ 1 Sekunde auf moderner Hardware.

10.3.8 Chemoinformatik: Molekül-Substruktur-Suche

Beispiel 10.9 (Pharmawirkstoff-Screening). Ein Pharmaunternehmen sucht in einer Datenbank von 10^6 Molekülen (als Graphen: Atome = Knoten, chemische Bindungen = Kanten) nach solchen, die ein bestimmtes pharmazeutisch aktives Teilmotiv $G_{\text{Pharmakophor}}$ als Subgraph enthalten.

Für $n = 30$ Atome im Pharmakophor-Graphen und typische Moleküle mit $N = 100$ Atomen:

$$\begin{aligned} T_{\text{pro Molekül}} &= \mathcal{O}(30^3) = 27.000 \text{ Op.} \\ T_{\text{ges}} &= 10^6 \cdot 27.000 = 2.7 \times 10^{10} \text{ Op.} \end{aligned}$$

Auf einem modernen Server (10^{10} Operationen/s): ~ 3 Sekunden für die gesamte Datenbank. Mit naivem Isomorphismus ($30! \approx 10^{32}$) wäre dies astronomisch unmöglich.

10.4 Numerisches Vergleichsbeispiel

Wir führen den Algorithmus vollständig für ein konkretes Beispiel durch:

Beispiel 10.10 (Vollständiger Algorithmus-Lauf). Gegeben:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Schritt 1 – Signaturen von A :

$$\sigma^A = [1, 18, 36, 48].$$

(Beispiel Spalte 1: $0 \cdot 2^0 + 1 \cdot 2^1 = 2$, Positionsgewicht $1 \cdot 2^4 = 16$, Summe = 18.)

Schritt 2 – Signaturen von B :

$$\sigma^B = [5, 18, 36, 48].$$

(Spalte 0: $[0, 0, 0, 0]^T + [1, 0, 0, 0]^T$; Zeilen-Hash: $2^0 = 1$ und $2^2 = 4$, Summe = 5.)

Schritt 3 – Rotations-Match ($A \subseteq B?$):

Zeilen-Hashes: $r^A = [1, 2, 4, 0]$, $r^B = [5, 2, 4, 0]$.

– Rotation 0: $\text{LCS}([1, 2, 4, 0], [5, 2, 4, 0]) = 3 \geq 2 \Rightarrow \text{Treffer}$.

Schritt 4 – Rotations-Match ($B \subseteq A?$):

$r^B = [5, 2, 4, 0]$. Alle 4 Rotationen liefern $\text{LCS} \leq 2$ mit Zeilen-Hash 5 (Spalte 0 von B) ohne Entsprechung in A . $\Rightarrow$ kein Treffer.

Ergebnis: $A \subseteq B$ aber $B \not\subseteq A \Rightarrow \text{keep_B}$.

G' enthält alle Kanten von G plus eine zusätzliche $((v_0, v_2))$ – G' hat mehr Information.

10.5 Zusammenfassung

Kernbotschaft des Subgraph Algorithmus:

1. **Signatur-Hashing** kodiert jede Matrixspalte eindeutig in einer Zahl – injektiv, kollisionsfrei, in $\mathcal{O}(n)$ pro Spalte.
2. **Zyklische Rotation** reduziert $n!$ auf n zu prüfende Knotenordnungen – ein Sprung von faktorieller zu linearer Suchtiefe.
3. **LCS-Vergleich** in $\mathcal{O}(n^2)$ erkennt gemeinsame Teilstrukturen robust und vollständig.
4. **Gesamtlaufzeit** $\mathcal{O}(n^3)$ ist bedingt optimal (unter SETH): Keine Verbesserung auf $o(n^3)$ ist möglich, ohne SETH zu widerlegen.
5. **Sieben Anwendungsdomänen** – Software, Bioinformatik, Genetik, Neurowissenschaften, Regelungstechnik, Finanzen, Chemie – zeigen den universellen Nutzen dieses Ansatzes.

11 Warum Graphen universell optimal sind

11.1 Das Prinzip der minimalen universellen Struktur

Wir haben in vier Beweisen und sechs Anwendungsdomänen gezeigt:

- P1. Informationstheorie:** Die binäre Beziehung (Kante) ist die minimale informationstheoretische Einheit. Graphs erreichen die Shannon-Schranke.
- P2. Algorithmik:** Divide-and-Conquer auf Graphen erzeugt $\mathcal{O}(\log n)$ -Algorithmen. Dies ist optimal nach dem Entscheidungsbaum-Modell.

- P3. Repräsentation:** Jede endliche Relationenstruktur ist isomorph zu einem Graphen. Kein Ausdruckskraft geht verloren.
- P4. Kommunikation:** Graph-Netzwerke dieser Struktur (Small-World, Scale-Free, Expander) minimieren Kommunikationsdistanz und -kosten.
- P5. Stabilität:** Spektrale Grapheigenschaften (λ_2 , Cheeger-Konstante) quantifizieren Robustheit. Das Optimierungsproblem (OSA) ist effizient lösbar.
- P6. Universalität:** Von der Primzahlarithmetik über das Konnektom bis zum Finanzsystem – die Graphstruktur erscheint in jeder Wissenschaft natürlich.

Satz 11.1 (Universeller Optimalitätssatz). *Sei Σ eine beliebige endliche Struktur über einer n -elementigen Domäne mit binären Relationen. Dann ist die Graphdarstellung von Σ die eindeutig optimale Darstellungsform im Sinne von:*

- (i) *Minimaler Beschreibungskomplexität ($\mathcal{O}(n+m)$ statt $\mathcal{O}(n^2)$ für dichte Matrizen oder $\mathcal{O}(n!)$ für Permutationsdarstellungen),*
- (ii) *Maximaler algorithmischer Effizienz ($\mathcal{O}(\log n)$ Suche, $\mathcal{O}(n \log n)$ Sortierung, $\mathcal{O}(n+m)$ Traversal),*
- (iii) *Minimaler Kommunikationskomplexität (Expander-Graphen erzielen $\mathcal{O}(\log n)$ Diameter bei linearer Kantenanzahl),*
- (iv) *Maximaler Robustheit gegenüber lokalem Ausfall (Fiedler-Wert bleibt hoch nach Entfernung von $o(\sqrt{n})$ Knoten in gut-vernetzten Graphen).*

12 Fazit und Ausblick

12.1 Zusammenfassung der Hauptergebnisse

Diese Arbeit hat gezeigt:

1. **Formaler Beweis der logarithmischen Optimalität:** Die Graphmodellierung erreicht die informationstheoretische Untergrenze $\mathcal{O}(\log n)$ für Suche, Kommunikation und Beschreibung (Sätze 3.1 to 3.3).
2. **Universalitätsbeweis:** Jede endliche binäre Relationsstruktur ist isomorph zu einem Graphen (Satz 3.4). Damit ist die Graphmodellierung verlustfrei universell.
3. **Interdisziplinäre Verifikation:** In sechs Disziplinen (Mathematik, Genetik, Neurowissenschaft, Regelungstechnik, Finanzwesen) wurde gezeigt, dass Graphen die natürliche und optimale Modellierungsform für die jeweiligen Strukturen sind.

12.2 Offene Fragen

1. **Hypergraphen und höhere Ordnungen:** Können Hyperkanten (Relationen ≥ 3 -stellig) in das logarithmische Optimierungsrahmenwerk integriert werden? Erste Ergebnisse deuten auf $\mathcal{O}(\log^k n)$ für k -uniforme Hypergraphen hin.

2. **Dynamische Graphen:** In Echtzeit veränderliche Graphen (soziale Netze, Handelsnetzwerke) erfordern dynamische OSA-Varianten. Ziel: amortisiert $\mathcal{O}(\log^2 n)$ pro Update.
3. **Quantengraphen:** Quantennetzwerke (Quantenkryptographie, -kommunikation) lassen sich als gewichtete Graphen mit komplexen Kantengewichten modellieren. Das Spektrum der Laplace-Matrix entspricht dann dem Energiespektrum des Quantengraphen.
4. **Kausalität:** Kausale Inferenz (Pearl’sche DAGs) verbindet Graphenstruktur mit statistischer Identifizierbarkeit. Eine vollständige logarithmische Theorie der kausalen Entdeckungsalgorithmen steht noch aus.

12.3 Ausblick

Die vorliegende Arbeit hat den Beweis erbracht, dass Graphen die mathematisch optimale Modellierungsform für alle endlichen Relationsstrukturen sind. Dieser Befund öffnet eine breite Schneise an Forschungsfragen, die in den kommenden Jahren und Jahrzehnten bearbeitet werden müssen. Die folgenden Abschnitte skizzieren diese Richtungen ausführlich – von unmittelbar anstehenden algorithmischen Erweiterungen bis hin zu tiefgreifenden epistemologischen Fragen über die Natur wissenschaftlicher Modellierung selbst.

12.4 Erweiterungen der logarithmischen Graphentheorie

12.4.1 Logarithmische Hierarchien in Hypergraphen

Die vorliegende Arbeit beschränkt sich auf binäre Relationen (Kanten zwischen je zwei Knoten). Viele reale Systeme erzeugen jedoch Beziehungen höherer Stelligkeit: chemische Reaktionen zwischen drei oder mehr Molekülen, soziale Gruppen-Interaktionen, biologische Komplexe aus mehreren Proteinen.

Ein k -uniformer Hypergraph $H = (V, \mathcal{E})$ besitzt Hyperkanten $e \in \mathcal{E}$ mit $|e| = k$. Die zentrale offene Frage lautet:

Forschungsfrage H1: Gilt für k -uniforme Hypergraphen die logarithmische Optimierungsschranke $T_{\text{opt}}(n) = \mathcal{O}(\log^k n)$? Und existiert ein Hypergraph-OSA, der diese Schranke erreicht?

Erste Ansätze existieren in der Tensorzerlegung [34]: Ein k -uniformer Hypergraph kann als Tensor $\mathcal{A} \in \mathbb{R}^{n \times \dots \times n}$ (k -fach) dargestellt werden. Die Spektraltheorie solcher Tensoren [40] verallgemeinert den Fiedler-Wert und könnte die Grundlage eines Hypergraph-OSA bilden. Die erwartete Laufzeit liegt bei $\mathcal{O}(n^{k-1} \log^k n)$, was für $k = 2$ genau den OSA reproduziert.

Die praktische Bedeutung ist enorm: In der Chemoinformatik [21] sind molekulare Reaktionsgraphen inhärent hypergraphisch; in der Neurobiologie bilden synaptische Ensembles keine Zweier-, sondern Gruppenverbindungen [41].

12.4.2 Kontinuierliche Graphen und Graphon-Theorie

Für sehr große oder unendliche Graphen ($n \rightarrow \infty$) wird die diskrete Graphstruktur durch *Graphons* ersetzt – messbare Funktionen $W : [0, 1]^2 \rightarrow [0, 1]$, die den Kontinuumslimites bilden [36].

Die logarithmische Struktur überträgt sich: Die Eigenwertverteilung des Graphon-Operators entspricht dem Limit des diskreten Laplace-Spektrums. Der Fiedler-Wert konvergiert zu einer spektralen Lücke im kontinuierlichen Operator. Offene Forschungsfragen sind:

- Wie konvergiert der OSA auf Graphon-Approximationen?
- Lässt sich der universelle Optimalitätssatz (Satz 11.1) auf Graphons verallgemeinern?
- Welche topologischen Invarianten bleiben im Graphon-Limes erhalten?

12.4.3 Persistente Homologie und topologische Graphanalyse

Die Verbindung zwischen Graphentheorie und algebraischer Topologie ist durch die *Persistente Homologie* [26] besonders fruchtbar geworden. Für eine Folge von Graphen $G_0 \subseteq G_1 \subseteq \dots \subseteq G_m$ (erzeugt durch schrittweise Hinzunahme von Kanten mit steigendem Gewicht) entsteht ein *Persistenz-Barcode*, der topologische Merkmale (verbundene Komponenten, Zyklen, Löcher) über alle Schwellwerte hinweg verfolgt.

Die Länge einer Persistenz-Bar ist logarithmisch in der Anzahl kritischer Ereignisse, was die Verbindung zur logarithmischen Graphmodellierung herstellt. Für die Biologie verspricht dies:

1. Robuste Charakterisierung von Proteinstruktur-Netzwerken gegenüber Messrauschen,
2. Identifikation essentieller Ringstrukturen in metabolischen Pfaden [52],
3. Frühzeitige Erkennung topologischer Veränderungen im Konnektom bei neurodegenerativen Erkrankungen.

12.5 Quantencomputing und Quantengraphentheorie

12.5.1 Quantenwalk und logarithmische Suche

Der klassische Random Walk auf einem Graphen braucht im schlechtesten Fall $\mathcal{O}(n)$ Schritte, um alle Knoten zu besuchen. Der *Quantenwalk* [19] überlagert alle Pfade kohärent und erreicht quadratische Beschleunigung: $\mathcal{O}(\sqrt{n})$ für unstrukturierte Suche. Auf gut strukturierten Graphen (Expander, Gitter) verbessert sich dies weiter auf $\mathcal{O}(\log n)$ – exakt die logarithmische Schranke dieser Arbeit, diesmal aber im Quantenregime.

Der *Quantum Graph Neural Network (QGNN)*-Ansatz [50] kodiert Graphzustände als Quantenschaltkreise. Die Tiefe des Quantenschaltkreises entspricht dem Durchmesser des Graphen – logarithmisch für Small-World-Graphen. Diese Verbindung zwischen Graphstruktur und Quantenverarbeitungseffizienz ist ein aktives Forschungsgebiet mit enormem Potenzial.

12.5.2 Quantennetzwerke als gewichtete Graphen

Quantenkommunikationsnetzwerke (Quanten-Internet) lassen sich als gewichtete Graphen modellieren, wobei Kantengewichte die Entanglement-Fidelity zwischen Quantenknoten kodieren [32]. Das Routing-Problem (finde den Pfad maximaler Fidelity zwischen Sender und Empfänger) ist ein klassisches kürzeste-Pfad-Problem auf diesem gewichteten Graphen.

Die Besonderheit: Quantenkanten können *gleichzeitig* mehrere Zustände kodieren (Superposition), was den effektiven Graphdurchmesser auf $\mathcal{O}(\log n)$ reduziert. Ein vollständiger Quanteninternet-Graph über n Knoten könnte so mit nur $\mathcal{O}(n \log n)$ physikalischen Verbindungen Vollkonnektivität simulieren – logarithmische Effizienz durch Quantenüberlagerung.

12.5.3 Variational Quantum Eigensolver auf Graphen

Der Variational Quantum Eigensolver (VQE) [39] löst Optimierungsprobleme auf Molekülgraphen. Der Hamiltonoperator des Moleküls wird als Matrix über dem Graphen der Atomorbitale geschrieben; VQE findet den Grundzustand durch Optimierung auf einer Quantenschaltkreis- Parametrisierung, deren Struktur selbst ein Graph ist. Die Konvergenz ist logarithmisch in der Energiepräzision – ein weiterer Beleg für die universelle logarithmische Natur graphenbasierter Optimierung.

12.6 Künstliche Intelligenz und Graphen

12.6.1 Graph Neural Networks: Theorie und Grenzen

Graph Neural Networks (GNNs) [33, 48] haben in der letzten Dekade revolutionäre Ergebnisse in Moleküldesign, Wissensrepräsentation und kombinatorischer Optimierung erzielt. Das Grundprinzip folgt direkt dem Divide-and-Conquer-Prinzip: Jeder Knoten aggregiert Information von seinen Nachbarn (lokale Halbierung), schichtet dies über L Ebenen (logarithmische Tiefe) und bildet so eine globale Darstellung.

Theoretisch wurde gezeigt [55], dass die Ausdrucksstärke von GNNs durch den Weisfeiler-Leman-Graphisomorphismustest begrenzt ist. Offene Frage: Wie können GNNs diese Grenze durch höherwertige Nachbarschaftsaggregation (analog zu Hyperkanten) überwinden, ohne die logarithmische Tiefeneffizienz zu verlieren?

12.6.2 Transformer als vollständige Graphen

Transformer-Architekturen [47] – die Grundlage aller großen Sprachmodelle – sind informell vollständige gewichtete Graphen: jedes Token (Knoten) ist mit jedem anderen durch einen Aufmerksamkeitsmechanismus (Kante mit Gewicht = Attention-Score) verbunden. Die Softmax-Normalisierung erzeugt ein Wahrscheinlichkeitsmaß über alle Kanten.

Die Tiefe des Transformers (L Schichten) entspricht der logarithmischen Pfadtiefe: mit $L = \mathcal{O}(\log n)$ Schichten kann ein Transformer jede boolesche Schaltkreis- berechnung der Tiefe $\log n$ simulieren [38]. Dies erklärt, warum Transformer mit ~ 32 – 96 Schichten so mächtig sind: sie realisieren bis zu 2^{96} kompositionelle Konzepte.

12.6.3 Kausal-AI und Interventionsgraphen

Pearl’s *Do-Calculus* [16] modelliert kausale Interventionen ($do(X = x)$) als Graph-Chirurgie: Kanten zum intervenierten Knoten werden entfernt. Die kausale Identifizierbarkeit einer Fragestellung ist äquivalent zur graphentheoretischen Bedingung *d-Separation* im kausalen DAG [51].

Die Forschungsrichtung *Causal Representation Learning* [44] versucht, kausale Graphstrukturen direkt aus Rohdaten zu lernen. Die logarithmische Optimierung tritt hier in Form des *Greedy Equivalence Search* [22] auf: der Algorithmus traversiert den Raum aller Graphen mit $\mathcal{O}(\log n)$ amortisierten Kanten-Modifikationen pro Schritt.

12.6.4 Reinforcement Learning auf Graphen

Viele Reinforcement-Learning-Probleme (Robotik, Spieltheorie, Ressourcenzuweisung) besitzen natürliche Graphstruktur: Zustände sind Knoten, Aktionen sind Kanten, Bellman-Gleichungen definieren eine rekursive Wertfunktion über den Graphen. Der Q-Learning-Algorithmus [53] konvergiert in $\mathcal{O}(|V| \cdot |E| \cdot \log(1/\epsilon))$ Schritten – logarithmisch in der gewünschten Genauigkeit. Graph-basierte Policy-Darstellungen versprechen eine weitere logarithmische Beschleunigung durch strukturelle Ausnutzung.

12.7 Biowissenschaften: Vom Molekül zum Ökosystem

12.7.1 Single-Cell-Genomik als Graphproblem

Die Single-Cell-RNA-Sequenzierung (scRNA-seq) erzeugt für jeden der 10^4 – 10^6 individualisierten Zellen einen Genexpressionsprofil-Vektor in ~ 20.000 -dimensionalem Raum. Die Zell-zu-Zell-Ähnlichkeit wird als k -NN-Graph modelliert [54], auf dem Clusteralgorithmen (Louvain, Leiden) Zelltypen identifizieren.

Die logarithmische Effizienz tritt mehrfach auf:

1. UMAP-Dimensionsreduktion [37] baut intern einen approximativen k -NN-Graphen in $\mathcal{O}(n \log n)$.
2. Trajektorienanalyse (Pseudozeit) entspricht dem Problem des kürzesten Pfads im Zell-Graphen.
3. Regulon-Erkennung (SCENIC [18]) sucht dichte Subgraphen im Kopräsenz-Graphen von Transkriptionsfaktoren und Zielgenen.

12.7.2 Ökologische Nahrungsnetze und Resilienz

Ein ökologisches *Nahrungsnetz* ist ein gewichteter Digraph $G_{\text{Eco}} = (V_{\text{Art}}, E_{\text{Fraß}}, w_{\text{Biomasse}})$. Die topologische Robustheit dieses Graphen gegenüber Artensterben (Knotenausfall) bestimmt die ökosystemische Resilienz [24].

Forschung hat gezeigt, dass reale Nahrungsnetze eine Mischung aus einer Baum-artigen Hierarchie (trophische Ebenen = log-Tiefe) und zyklischen Omnivorie-Kanten aufweisen. Der Verlust von Schlüsselarten (Keystone Species = Graph-Hubs) erhöht den effektiven Graphdurchmesser dramatisch – analog zum Alzheimer-Effekt aus Abschnitt 6. Der OSA kann Keystone Species als subgraph-maximierende Knoten identifizieren, bevor ihr Verlust das Ökosystem destabilisiert.

12.7.3 Epigenetik: Chromatinnetzwerke

Das dreidimensionale Genom ist ein Graph: DNA-Schleifen und Chromatindomänen (TADs) bilden Knoten, räumliche Kontakte (gemessen durch Hi-C-Experimente [35]) bilden Kanten. Die Graphstruktur bestimmt, welche Gene miteinander in regulatorischen Kontakt treten – und damit, welche Gene gleichzeitig aktiviert oder stillgelegt werden.

Das Hi-C-Kontaktnetzwerk zeigt eine ausgeprägte Block-Diagonal-Struktur (TADs), die einer hierarchischen Graphpartitionierung entspricht: Die Bisektionsmethode des OSA ist formal äquivalent zu TAD-Identifikationsalgorithmen. Tiefe berechnet sich mit $\text{Tiefe} = \log_2(\text{Genomlänge}/\text{TAD-Größe}) \approx \log_2(3 \times 10^9/10^5) \approx 15$ – logarithmisch, wie erwartet.

12.7.4 Strukturbilogie: Protein-Faltung als Graphproblem

AlphaFold2 [31] – der Durchbruch in der Proteinfaltungsvorhersage – modelliert das Protein als *Residue Contact Graph*: Aminosäuren sind Knoten, räumliche Nachbarschaft ($< 8 \text{ \AA}$) sind Kanten. Der Transformer-Kern von AlphaFold2 propagiert Information über diesen Graphen in logarithmisch vielen Schichten. Die spektrale Analyse des Kontaktgraphen [49] erlaubt eine Vorhersage der Faltungskernresiduen – diejenigen Knoten, die den Fiedler-Vektor dominieren.

12.8 Physik, Klimaforschung und Komplexe Systeme

12.8.1 Phasengraphen in der statistischen Mechanik

Das *Ising-Modell* – das Standardmodell für Phasenübergänge – ist auf einem Gittergraphen definiert: Spins sind Knoten, Wechselwirkungen sind Kanten. Der Phasenübergang entspricht einem *Perkulationsübergang* im zugehörigen Zufallsgraphen [28]. Die kritische Perkulationsschwelle ist direkt mit dem Fiedler-Wert λ_2 korreliert: Der Übergang geschieht genau dann, wenn $\lambda_2 \rightarrow 0$.

Dies eröffnet die Perspektive, Phasenübergänge universell als spektrale Graph-Ereignisse zu beschreiben – ein Rahmenwerk, das von der Physik über die Neurologie (epileptische Anfälle = Synchronisationsübergang [46]) bis zur Ökologie (Kollaps-Tipping-Points) reicht.

12.8.2 Klimanetzwerke und Tipping Elements

Das Erdsystem wird als *Klimanetzwerk* modelliert [23]: Klimastationen (oder Gitterzellen) sind Knoten, statistische Telekonnektionen (verzögerte Korrelationen in Zeitreihen) sind Kanten. El-Niño-Ereignisse erscheinen als rapide Umstrukturierungen der Hub-Knoten im Pazifik.

Der OSA erlaubt die Identifikation von *Tipping-Element-Netzwerken*: Teilgraphen, deren internes Feedback (hohes λ_2 , hohe Dichte) sie zu stabilen Lock-in-Zuständen prädisponiert. Kippunkte des Klimasystems (Arktiseis-Schmelze, Amazonas-Dieback, AMOC-Kollaps) sind graphentheoretisch als Bifurkationen im Subgraph-Spektrum beschreibbar.

12.8.3 Topologische Quantenfeldtheorie und Spin-Netzwerke

In der Loop-Quantengravitation [42] wird die Raumzeit selbst als *Spin-Netzwerk* modelliert – ein kombinatorischer Graph, dessen Knoten Raum-Volumen-Quanten und dessen Kanten Flächen-Quanten tragen. Dies ist die radikalste mögliche Anwendung der Graphmodellierung: nicht nur reale Objekte in der Raumzeit werden als Graphen dargestellt, sondern die Raumzeit selbst ist ein Graph. Die Planck-Skala ($\sim 10^{-35} \text{ m}$) entspricht der atomaren Grapheneinheit – G_{atom} als Fundament des Universums.

12.9 Gesellschaft, Recht und digitale Infrastruktur

12.9.1 Wissensgraphen und semantisches Web

Das *semantische Web* [20] organisiert das gesamte Weltwissen als gewichteten Hypergraph (RDF-Tripel: Subjekt–Prädikat–Objekt). Google’s Knowledge Graph umfasst > 500 Milliarden Fakten als Graphkanten. Sprachmodelle wie GPT-4 oder Gemini greifen implizit

auf derartige Graphstrukturen zurück – die Aufmerksamkeitsmatrix ist ein vollständiger, dynamisch gewichteter Graph über alle Token-Knoten.

Die Suche in Wissensgraphen (Graphen-RAG [25]) profitiert direkt von der logarithmischen Pfadlänge: Selbst in einem Wissensgraphen mit 10^{12} Knoten beträgt die mittlere Pfadlänge zwischen zwei Konzepten nur $\sim \log_{10}(10^{12}) = 12$ Schritte.

12.9.2 Rechtliche und ethische Implikationen großer Graphen

Mit zunehmender Durchdringung gesellschaftlicher Entscheidungsprozesse durch graphenbasierte Algorithmen (Kreditwürdigkeitsgraphen, soziale Empfehlungsgraphen, gerichtliche Präzedenzfallnetzwerke) entstehen neue rechtliche und ethische Fragen:

- **Erklärbarkeit:** Logarithmische Baumstrukturen erlauben Entscheidungspfade der Tiefe $\mathcal{O}(\log n)$, die als Entscheidungsbaum vollständig erklärbar sind – im Gegensatz zu flachen neuronalen Netzen.
- **Fairness:** Knotengrad-Ungleichheit in Empfehlungsgraphen erzeugt systematische Benachteiligung von Niedrig-Grad-Knoten. Graphentheoretische Fairness-Metriken [43] müssen entwickelt werden.
- **Datenschutz:** Linkage-Angriffe auf anonymisierte Graphen nutzen die logarithmisch kleine Diameter-eigenschaft aus. Differential Privacy auf Graphen [29] ist ein aktives Forschungsfeld.

12.9.3 Internet der Dinge und Edge Computing

IoT-Netzwerke aus Milliarden vernetzter Geräte bilden dynamische Graphen mit sich ständig ändernder Topologie. Edge-Computing-Strategien [45] partitionieren diesen Graphen rekursiv (analog zum OSA), um Latenz zu minimieren und Bandbreite zu sparen. Das optimale Partitionierungsproblem – welche Knoten lokal verarbeiten, welche auslagern – ist formal identisch mit dem Minimalen-Schnitt-Problem auf dem IoT-Graphen. Der Fiedler-Vektor liefert die optimale Lösung in $\mathcal{O}(n \log n)$.

12.10 Transdisziplinäre Perspektiven und Metamodellierung

12.10.1 Graphen als Wissenschaftssprache

Diese Arbeit hat gezeigt, dass Graphen nicht nur in einzelnen Disziplinen nützlich sind, sondern eine universelle *Wissenschaftssprache* bilden. Die nächste Forschungsgeneration steht vor der Aufgabe, eine formale *Graphengrammatik der Wissenschaften* zu entwickeln – eine Theorie, die beschreibt, wie Modelle verschiedener Disziplinen durch Graph-Morphismen [27] ineinander übersetzbar sind.

Ein erster Schritt ist die *kategorientheoretische* Formulierung: Graphen sind Objekte in der Kategorie **Graph**, Morphismen sind strukturerhaltende Abbildungen. Funktoren zwischen Kategorien erlauben den formalen Transfer von Resultaten zwischen Disziplinen – z.B. von der algebraischen Topologie (Spektralsequenzen) in die Biologie (Entwicklungshierarchien).

12.10.2 Komplexitätstheorie und untere Schranken

Eine fundamentale offene Frage ist, ob die in dieser Arbeit bewiesene logarithmische Schranke $\mathcal{O}(\log n)$ wirklich optimal ist oder ob sub-logarithmische Verfahren für spezielle Graphklassen existieren. Die theoretische Informatik kennt für bestimmte Suchprobleme $\Omega(\log n)$ als beweisbar optimale untere Schranke [30]. Es bleibt zu klären:

1. Für welche Graphklassen ist $o(\log n)$ erreichbar?
2. Welche algebraischen Eigenschaften (Regularität, Symmetrie, Expansionskonstante) ermöglichen sub-logarithmische Algorithmen?
3. Gibt es einen allgemeinen Trade-off zwischen Graphstruktur (Komplexität der Kanten) und algorithmischer Tiefe (Komplexität des Algorithmus)?

12.10.3 Interdisziplinäre Bildung: Den Graphen ins Zentrum rücken

Schließlich hat die vorliegende Arbeit eine pädagogische Implikation: Graphentheorie sollte als *Grundlagenfach* in allen naturwissenschaftlichen und ingenieurwissenschaftlichen Studiengängen verankert werden. Wer Graphen verstehen kann – ihre Struktur, ihr Spektrum, ihre algorithmische Verarbeitung – ist mit einem universellen Werkzeug ausgestattet, das in jedem Fachgebiet sofort einsetzbar ist.

Die logarithmische Sichtweise – das Problem zu halbieren, die Struktur zu teilen, und dann rekursiv zu verfeinern – ist nicht nur ein Algorithmus. Es ist eine Denkweise. Sie ist die Essenz des wissenschaftlichen Fortschritts: Ordnung durch Aufteilung, Verstehen durch Verknüpfung, Erkenntnis durch Traversierung.

12.11 Schlusswort

Graphen sind nicht bloß ein nützliches Werkzeug. Sie sind – wie diese Arbeit beweist – die *notwendige* und *optimale* Sprache, in der endliche diskrete Strukturen beschrieben werden müssen, wenn man Effizienz, Vollständigkeit und Robustheit zugleich maximieren will. Die Elementareinheit zweier Knoten und einer Kante ist der Urquell dieser Stärke: Sie teilt das Problem, halbiert den Raum, erzeugt den Logarithmus – und wiederholt dies so lange, bis Ordnung aus Komplexität entsteht.

Literatur

- [1] Barabási, A.-L. & Albert, R. (1999). *Emergence of Scaling in Random Networks*. Science, 286(5439), 509–512.
- [2] Bollobás, B. (1998). *Modern Graph Theory*. Springer, New York.
- [3] Chung, F. R. K. (1997). *Spectral Graph Theory*. American Mathematical Society.
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press, Cambridge.
- [5] Felleman, D. J. & Van Essen, D. C. (1991). Distributed Hierarchical Processing in the Primate Cerebral Cortex. *Cerebral Cortex*, 1(1), 1–47.

- [6] Gardner, T. S., Cantor, C. R., & Collins, J. J. (2000). Construction of a genetic toggle switch in *Escherichia coli*. *Nature*, 403, 339–342.
- [7] Goldberg, A. V. (1984). *Finding a Maximum Density Subgraph*. Technical Report UCB/CSD-84-171, UC Berkeley.
- [8] Mantegna, R. N. (1999). Hierarchical Structure in Financial Markets. *European Physical Journal B*, 11(1), 193–197.
- [9] Montufar, G. F., Pascanu, R., Cho, K., & Bengio, Y. (2014). On the Number of Linear Regions of Deep Neural Networks. *Advances in Neural Information Processing Systems*, 27.
- [10] Newman, M. E. J. (2003). The Structure and Function of Complex Networks. *SIAM Review*, 45(2), 167–256.
- [11] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379–423.
- [12] Spielman, D. A. & Teng, S.-H. (1996). Spectral Partitioning Works: Planar Graphs and Finite Element Meshes. *Proceedings of FOCS 1996*, 96–105.
- [13] Watts, D. J. & Strogatz, S. H. (1998). Collective Dynamics of ‘Small-World’ Networks. *Nature*, 393, 440–442.
- [14] West, D. B. (2001). *Introduction to Graph Theory* (2nd ed.). Prentice Hall, Upper Saddle River.
- [15] Diestel, R. (2017). *Graph Theory* (5th ed.). Springer, Heidelberg.
- [16] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.
- [17] Epp, S. (2026). *Der Subgraph Algorithmus*. Verfügbar unter: <https://gitlab.com/epp-group/subgraph>.
- [18] Aibar, S., González-Blas, C. B., Moerman, T., Huynh-Thu, V. A., Imrichová, H., Hulselmans, G., ... & Aerts, S. (2017). SCENIC: single-cell regulatory network inference and clustering. *Nature Methods*, 14(11), 1083–1086.
- [19] Aharonov, D., Ambainis, A., Kempe, J., & Vazirani, U. (2001). Quantum Walks on Graphs. *Proceedings of the 33rd Annual ACM STOC*, 50–59.
- [20] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34–43.
- [21] Bertz, S. H. (1981). The first general index of molecular complexity. *Journal of the American Chemical Society*, 103(12), 3599–3601.
- [22] Chickering, D. M. (2002). Optimal Structure Identification with Greedy Search. *Journal of Machine Learning Research*, 3, 507–554.
- [23] Donges, J. F., Zou, Y., Marwan, N., & Kurths, J. (2009). The backbone of the climate system. *Geophysical Research Letters*, 36(18), L18705.

- [24] Dunne, J. A., Williams, R. J., & Martinez, N. D. (2002). Network Structure and Biodiversity Loss in Food Webs. *Ecology Letters*, 5(4), 558–567.
- [25] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., . . . & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*.
- [26] Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2002). Topological Persistence and Simplification. *Discrete & Computational Geometry*, 28(4), 511–533.
- [27] Ehrig, H., Ehrig, K., Prange, U., & Taentzer, G. (2006). *Fundamentals of Algebraic Graph Transformation*. Springer, Berlin.
- [28] Erdős, P. & Rényi, A. (1960). On the Evolution of Random Graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, 5, 17–61.
- [29] Hay, M., Miklau, G., Jensen, D., Weis, P., & Srivastava, S. (2009). Anonymizing Social Networks. *Technical Report 07-19*, University of Massachusetts.
- [30] Hromkovič, J. (2004). *Algorithmics for Hard Problems* (2nd ed.). Springer, Berlin.
- [31] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., . . . & Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 583–589.
- [32] Kimble, H. J. (2008). The quantum internet. *Nature*, 453, 1023–1030.
- [33] Kipf, T. N. & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *Proceedings of ICLR 2017*.
- [34] Kolda, T. G. & Bader, B. W. (2009). Tensor Decompositions and Applications. *SIAM Review*, 51(3), 455–500.
- [35] Lieberman-Aiden, E., van Berkum, N. L., Williams, L., Imakaev, M., Ragoczy, T., Telling, A., . . . & Dekker, J. (2009). Comprehensive mapping of long-range interactions reveals folding principles of the human genome. *Science*, 326(5950), 289–293.
- [36] Lovász, L. (2012). *Large Networks and Graph Limits*. American Mathematical Society.
- [37] McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv preprint arXiv:1802.03426*.
- [38] Pérez, J., Marinković, J., & Barceló, P. (2021). On the Turing Completeness of Modern Neural Network Architectures. *Proceedings of ICLR 2019* (extended version).
- [39] Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., . . . & O’Brien, J. L. (2014). A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5, 4213.
- [40] Qi, L., & Luo, Z. (2005). Eigenvalues and invariants of tensors. *Journal of Mathematical Analysis and Applications*, 325(2), 1363–1377.

- [41] Reimann, M. W., Nolte, M., Scolamiero, M., Turner, K., Perin, R., Chindemi, G., ... & Markram, H. (2017). Cliques of Neurons Bound into Cavities Provide a Missing Link between Structure and Function. *Frontiers in Computational Neuroscience*, 11, 48.
- [42] Rovelli, C. (2004). *Quantum Gravity*. Cambridge University Press.
- [43] Saxena, N. A., Huang, K., DeFilippis, E., Radanovic, G., Parkes, D. C., & Liu, Y. (2022). How Do Fairness Definitions Fare? Examining Public Attitudes Towards Algorithmic Definitions of Fairness. *Proceedings of AAAI 2019* (extended).
- [44] Schölkopf, B., Locatello, F., Bauer, S., Ke, N. R., Kalchbrenner, N., Goyal, A., & Bengio, Y. (2021). Toward Causal Representation Learning. *Proceedings of the IEEE*, 109(5), 612–634.
- [45] Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, 3(5), 637–646.
- [46] Strogatz, S. H. (2001). Exploring Complex Networks. *Nature*, 410, 268–276.
- [47] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- [48] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph Attention Networks. *Proceedings of ICLR 2018*.
- [49] Vendruscolo, M., Dokholyan, N. V., Paci, E., & Karplus, M. (2002). Small-world view of the amino acids that play a key role in protein folding. *Physical Review E*, 65(6), 061910.
- [50] Verdon, G., McCourt, T., Lubasch, M., Leyton-Brown, K., Hidary, J., & Neven, H. (2019). Quantum Graph Neural Networks. *arXiv preprint arXiv:1909.12264*.
- [51] Verma, T. & Pearl, J. (1988). Causal Networks: Semantics and Expressiveness. *Proceedings of the 4th Workshop on Uncertainty in AI*, 352–359.
- [52] Wagner, A. & Fell, D. A. (2001). The small world inside large metabolic networks. *Proceedings of the Royal Society B*, 268(1478), 1803–1810.
- [53] Watkins, C. J. C. H. & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.
- [54] Wolf, F. A., Angerer, P., & Theis, F. J. (2018). SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1), 15.
- [55] Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How Powerful are Graph Neural Networks? *Proceedings of ICLR 2019*.